Companion to George Grätzer's

Vibe Computing for Mathematicians

Exercise Help and Solutions

Contents

How to Use this Companion v

1 Mathematics First 1
Mathematical Checkpoint 1

2 Arithmetic Investigations 7
Mathematical Checkpoint 7

3 Conjectures and Tests 11
Mathematical Checkpoint 11
Part I Capstone: A Worked Model 15

4 Trustworthy Mathematical Answers 17
Mathematical Checkpoint 17

5 Exact and Approximate Methods 21
Mathematical Checkpoint 21

6 Pictures, Tables, and Evidence 27
Mathematical Checkpoint 27
Part II Capstone: A Reusable Divisor Module 31

7 Sequences and Linear Algebra 33
Mathematical Checkpoint 33

8 Dynamics and Randomness 39
Mathematical Checkpoint 39

9 Graphs and Networks 45
Mathematical Checkpoint 45

10 Search and Ordered Structures 51

Mathematical Checkpoint 51
Part III Capstone: A Finite Ordered Set Analyzer 55

11 Testing and Proof 57
Mathematical Checkpoint 57

12 Reproducible Investigations 65
Mathematical Checkpoint 65
Part IV Capstone: A Completed Model 71

A Python: An Optional Introduction 73

How to Use this Companion

This Exercise Companion follows the exercises in *Vibe Computing for Mathematicians*. It covers all 83 numbered exercises in Chapters 1–12 and all 15 in Appendix A. It also answers the 12 mathematical checkpoints and gives worked models for the further investigations and the four part capstones.

Read the hint first, try the problem, and then consult the solution. Each solution explains the mathematical reasoning behind the answer. You can skip the Python listings and ask ChatGPT to run the supplied files. The separate *Python Programs* companion collects the complete programs from both books, including the two longer lattice routines, and identifies the chapter for each file. Where an exercise asks for a personal investigation, disclosure, or project, we give a model for you to adapt. Examples of AI exchanges are constructed for the exercise; they are not transcripts of actual conversations.

Exercise numbers have the form 7.3 or A.8: chapter followed by the item's number in that chapter. The chapter checkpoints and capstones keep their names. Mathematical conventions agree with the book: $F_0 = 0$, $F_1 = 1$; graphs are finite, undirected, and simple unless otherwise stated; orders are reflexive relations; operation tables are labelled and stored row by row.

The accompanying source package includes the programs used for verification, editable LaTeX, figures, and retained results. The file `programs/verify_solutions.py` recomputes the main numerical tables, plots, and exhaustive searches. The chapter example files contain the printed Python listings in reading order. Run these as complete files; later listings may use a function defined earlier in the chapter. Use Python without the `-O` option so that the assertions run.

Numerical tables describe the recorded computation. Last digits of floating-point results and long chaotic orbits can depend on the environment. Exact integer results do not have that qualification. The full environment is recorded with the results. Each solution makes clear whether its conclusion rests on a finite test or a proof.

Chapter 1
Mathematics First

Mathematical Checkpoint

Hint. Start the accumulator at zero and include the endpoint n.

Solution. The hand calculations give $0, 1, 9, 36$ for $n = 0, 1, 2, 3$. The extended comparison is

n	$\sum_{k=1}^{n} k^3$	$(\sum_{k=1}^{n} k)^2$
0	0	0
1	1	1
2	9	9
3	36	36
4	100	100
5	225	225
6	441	441
7	784	784
8	1296	1296
9	2025	2025
10	3025	3025

This suggests $\sum_{k=1}^{n} k^3 = [n(n+1)/2]^2$ for every $n \geq 0$. A proof is still needed. For induction, the difference between the proposed values at n and $n-1$ is $[n(n+1)/2]^2 - [(n-1)n/2]^2 = n^3$; the initial value is zero. The induction proves the identity for every nonnegative integer.

For a nonnegative integer n, the following returns $\sum_{k=1}^{n} k^3$.

```
def sum_cubes(n):
    if type(n) is not int or n < 0:
        raise ValueError("expected integer >= 0")
    total = 0
    for k in range(1, n + 1):
        total += k**3
    return total

assert [sum_cubes(n) for n in range(4)] == [0, 1, 9, 36]
print(sum_cubes(3))
for n in range(11):
```

```
    assert sum_cubes(n) == sum(range(n + 1))**2
    print(n, sum_cubes(n), sum(range(n + 1))**2)
```

The first printed answer is 36; the following rows give the full comparison. Python excludes the stop value of a range, so `range(1, n + 1)` includes n. Printing after the loop displays one final answer. At the start of the iteration for k, the accumulator is $\sum_{j=1}^{k-1} j^3$; adding k^3 preserves this description for the next iteration. The empty sum at $n = 0$ is zero.

Exercise 1.1. Two drawings of the Petersen graph

Hint. Separate the abstract adjacency relation from vertex positions.

Solution. Use the same labelled graph with two layouts.

```
import networkx as nx
import matplotlib.pyplot as plt
G = nx.petersen_graph()
fig, axes = plt.subplots(1, 2, figsize=(8, 4))
for ax, positions in zip(axes, [nx.circular_layout(G),
                                nx.spring_layout(G, seed=17)]):
    nx.draw_networkx(G, pos=positions, ax=ax,
                     node_color="white", edgecolors="black",
                     node_size=300, font_size=9)
    ax.set_axis_off()
fig.tight_layout()
plt.close(fig)
assert len(G) == 10 and G.number_of_edges() == 15
assert sorted(dict(G.degree()).values()) == [3] * 10
```

Both drawings have ten vertices, fifteen edges, degree three at every vertex, and the same paths, cycles, connectedness, and automorphisms. Connectedness, diameter, girth, chromatic number, and planarity are graph properties. Locations, edge lengths, apparent angles, and the number of crossings in a particular drawing depend on the layout. A crossing in one drawing alone does not prove nonplanarity.

Exercise 1.2. Primes below 200

Hint. A composite n has a divisor between 2 and $\lfloor \sqrt{n} \rfloor$.

Solution. The complete list is

$$2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47,$$
$$53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113,$$
$$127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199.$$

There are 46. To check the first ten, 2 and 3 need no smaller possible prime divisors; for 5 and 7 test 2; for $11, 13, 17, 19, 23$ test $2, 3$; and for 29 test $2, 3, 5$. None divides.

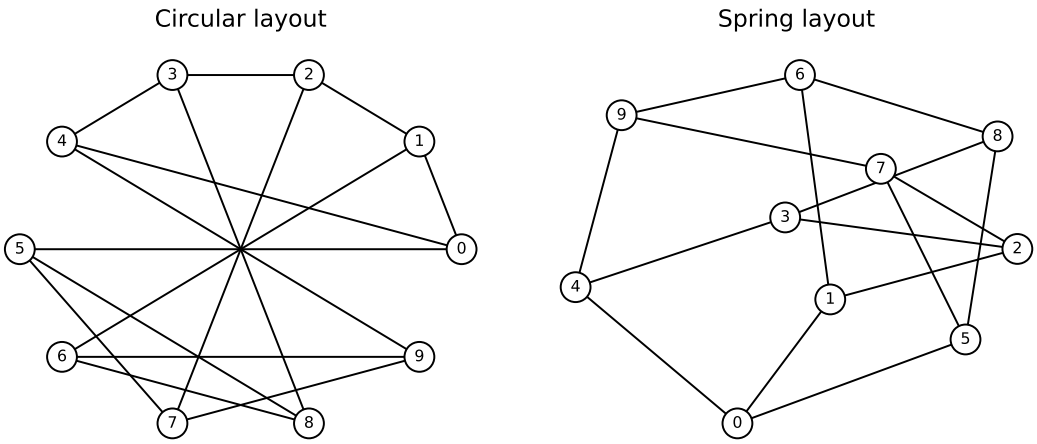


Fig. 1.1 The same labelled Petersen graph in circular and spring layouts.

```
from math import isqrt

def is_prime(n):
    if type(n) is not int:
        raise ValueError("expected integer")
    return n >= 2 and all(n % d != 0
                          for d in range(2, isqrt(n) + 1))

primes = [n for n in range(2, 200) if is_prime(n)]
assert len(primes) == 46
```

The square-root bound is justified because $n = ab$ with $a, b > 1$ cannot have both factors greater than $\sqrt{n}$.

Exercise 1.3. Fibonacci ratios

Hint. Generate the integers exactly; round only the final ratios.

Solution. Use $F_0 = 0, F_1 = 1$ and $F_n = F_{n-1} + F_{n-2}$.

```
f = [0, 1]
for _ in range(19):
    f.append(f[-1] + f[-2])
ratios = [(n, f[n] / f[n-1]) for n in range(5, 21)]
```

n	F_n	F_{n-1}	F_n/F_{n-1}
5	5	3	1.666666667
6	8	5	1.600000000
7	13	8	1.625000000
8	21	13	1.615384615
9	34	21	1.619047619
10	55	34	1.617647059
11	89	55	1.618181818
12	144	89	1.617977528
13	233	144	1.618055556
14	377	233	1.618025751
15	610	377	1.618037135
16	987	610	1.618032787
17	1597	987	1.618034448
18	2584	1597	1.618033813
19	4181	2584	1.618034056
20	6765	4181	1.618033963

The last ratio is approximately 1.618033963, and $\phi = (1 + \sqrt{5})/2 \approx 1.618033989$. The final five ratios, $n = 16$ through 20, all round to 1.61803 at five decimal places. Their unrounded decimal expansions share the prefix 1.61803; earlier rows have fewer stable digits. The apparent stabilization is a finite observation. The formula $F_n = (\phi^n - \psi^n)/\sqrt{5}$, where $\psi = (1 - \sqrt{5})/2$, proves convergence because $|\psi/\phi| < 1$.

Exercise 1.4. An independent check

Hint. Check a consequence by a route that does not repeat the original loop.

One model response. For the Petersen graph, inspect its construction: two five-cycles (the inner one drawn as a star) and five matching edges give 15 edges. Each of its ten vertices has degree three, so the handshaking identity also gives $2|E| = 10 \cdot 3 = 30$. This checks the reported edge count through a second representation. It would not, by itself, distinguish the Petersen graph from every other cubic graph on ten vertices; checking adjacency is stronger.

Further Investigation: A Worked Model

Hint. Choose a question whose first cases can be checked on paper.

One model response. Display $T_n = 1 + \cdots + n$ for $0 \le n \le 20$. The hand checks $T_3 = 6$ and $T_4 = 10$ agree with the program. An independent calculation uses $T_n = n(n+1)/2$, proved by pairing k with $n + 1 - k$ in two copies of the sum. The finite output lists 21 values; the pairing argument proves the formula for every nonnegative integer.

Chapter 2
Arithmetic Investigations

Mathematical Checkpoint

Hint. Exclude n itself, and return an empty list when $n = 1$.

Solution. For $n = 1$ the list is empty and its sum is zero. For $n = 12$, the list $1, 2, 3, 4, 6$ has sum 16. Complete enumeration gives

n	Proper positive divisors	Sum
1	$\varnothing$	0
2	1	1
3	1	1
4	$1, 2$	3
5	1	1
6	$1, 2, 3$	6
7	1	1
8	$1, 2, 4$	7
9	$1, 3$	4
10	$1, 2, 5$	8
11	1	1
12	$1, 2, 3, 4, 6$	16
13	1	1
14	$1, 2, 7$	10
15	$1, 3, 5$	9
16	$1, 2, 4, 8$	15
17	1	1
18	$1, 2, 3, 6, 9$	21
19	1	1
20	$1, 2, 4, 5, 10$	22
21	$1, 3, 7$	11
22	$1, 2, 11$	14
23	1	1

n	Proper positive divisors	Sum
24	$1, 2, 3, 4, 6, 8, 12$	36
25	$1, 5$	6
26	$1, 2, 13$	16
27	$1, 3, 9$	13
28	$1, 2, 4, 7, 14$	28
29	1	1
30	$1, 2, 3, 5, 6, 10, 15$	42

Only 6 and 28 equal their sums: $1 + 2 + 3 = 6$ and $1 + 2 + 4 + 7 + 14 = 28$. The displayed lists check both positive answers directly; the other rows exclude every remaining integer in the requested range.

A deliberately direct version is easy to inspect.

```
def proper_divisors(n):
    if type(n) is not int or n < 1:
        raise ValueError("expected positive integer")
    return [d for d in range(1, n) if n % d == 0]

assert proper_divisors(1) == []
assert proper_divisors(12) == [1, 2, 3, 4, 6]
print(sum(proper_divisors(12)))
for n in range(1, 31):
    ds = proper_divisors(n)
    print(n, ds, sum(ds))
assert [n for n in range(1, 31) if sum(proper_divisors(n)) == n] == [6, 28]
```

The calling code prints 16. Since the function returns a list, we can also count its entries, sum them, or display them without changing the search.

Exercise 2.1. Exactly six divisors

Hint. If $n = \prod p_i^{e_i}$, then its number of positive divisors is $\prod(e_i + 1)$.

Solution. The first ten integers are

$$12, 18, 20, 28, 32, 44, 45, 50, 52, 63.$$

The factorization of 6 shows that exactly two forms are possible: p^5, or p^2q for distinct primes p, q. Their factorizations are

$$2^2 3,\ 2 \cdot 3^2,\ 2^2 5,\ 2^2 7,\ 2^5,\ 2^2 11,\ 3^2 5,\ 2 \cdot 5^2,\ 2^2 13,\ 3^2 7.$$

```
answers = []
n = 1
while len(answers) < 10:
    if sum(n % d == 0 for d in range(1, n + 1)) == 6:
        answers.append(n)
    n += 1
assert answers == [12,18,20,28,32,44,45,50,52,63]
```

The search examines every positive integer through 63 in increasing order, establishing that these are the first ten.

Exercise 2.2. Euclid on $(391, 299)$

Hint. Retain the quotient and remainder in every division.

Solution.

$$\begin{aligned} 391 &= 1 \cdot 299 + 92, \\ 299 &= 3 \cdot 92 + 23, \\ 92 &= 4 \cdot 23 + 0. \end{aligned}$$

Thus the gcd is 23. Back-substitution gives $23 = 299 - 3(391 - 299) = 4 \cdot 299 - 3 \cdot 391$. Since 23 divides both inputs and is their integer linear combination, this is an independent certificate of the gcd.

Exercise 2.3. Cube residues modulo 7

Hint. Check the complete residue system $0, 1, \ldots, 6$.

Solution. The cubes in that order are $0, 1, 1, 6, 1, 6, 6$ modulo 7. The possible residues are therefore $\{0, 1, 6\} = \{0, 1, -1\}$ modulo 7. Every integer is congruent to exactly one of the checked representatives, and congruence is preserved by cubing, so this finite calculation proves the statement for all integers.

Exercise 2.4. A composite passing the base-two Fermat test

Hint. Look for a composite n with a short congruence such as $2^{10} \equiv 1 \pmod{n}$.

Solution. Take $n = 341 = 11 \cdot 31$. Since $2^{10} = 1024 = 3 \cdot 341 + 1$,

$$2^{340} = (2^{10})^{34} \equiv 1 \pmod{341}.$$

Both factors are greater than one; their product certifies compositeness. Also $\gcd(2, 341) = 1$, as required for the usual Fermat test.

```
assert 11 * 31 == 341
assert pow(2, 340, 341) == 1
```

Passing this test is a necessary condition for an odd prime, not a sufficient condition for primality.

Exercise 2.5. Evidence and conclusion

Hint. State a tested range before proposing a general theorem.

One model response. Compute the number $d(n)$ of positive divisors for $1 \le n \le 1000$. The evidence is that $d(n)$ is odd precisely at the squares in that range. The proposed theorem is: for every positive integer n, $d(n)$ is odd if and only if n is a square. Pair a divisor d with n/d. Distinct pairs contribute two divisors; an unpaired divisor exists precisely when $d = n/d$, that is, $d^2 = n$. This proves the theorem without relying on the finite range.

Further Investigation: A Worked Model

Hint. Use the pairing argument to design a second computation.

One model response. Investigate the same divisor-parity theorem. Return the triples $(n, d(n), \lfloor\sqrt{n}\rfloor^2 = n)$ for $1 \le n \le 1000$. Count divisors directly and also by paired divisors through the square root. Check 1, 9, and 12 by hand. Keep the full table and the tested range. Record any failures; there are none in this example. The pairing proof above turns the observed pattern into an unrestricted theorem.

Chapter 3
Conjectures and Tests

Mathematical Checkpoint

Hint. A successful answer can be zero; reserve `None` for failure.

Solution. The answers for [], $[1, 2, 4]$, and $[0, 1, 3]$ are, respectively, “no such entry”, “no such entry”, and 0. Zero is a multiple of 3, since $0 = 3 \cdot 0$, and it occurs first in the third list. We therefore need a different return value when no such entry exists.

```
def first_divisible(values, divisor):
    if type(divisor) is not int or divisor <= 0:
        raise ValueError("expected positive integer divisor")
    if any(type(x) is not int for x in values):
        raise ValueError("expected integer entries")
    for x in values:
        if x % divisor == 0:
            return x
    return None

assert first_divisible([], 3) is None
assert first_divisible([5, 0, 6], 3) == 0
answer = first_divisible([5, 0, 6], 3)
assert answer is not None
for values in [[], [1, 2, 4], [0, 1, 3]]:
    answer = first_divisible(values, 3)
    print("no such entry" if answer is None else answer)
assert first_divisible([1, 2, 4], 3) is None
assert first_divisible([0, 1, 3], 3) == 0
```

The expression `if answer` would treat zero as false. The correct success test is `answer is not None`. Scanning left to right and returning immediately proves that the returned entry is the first one.

Exercise 3.1. A difference table for $n^4 - n^2$

Hint. Begin at $n = 0$ and repeatedly subtract adjacent entries.

Solution.

n	0	1	2	3	4	5	6
$f(n)$	0	0	12	72	240	600	1260
Δf	0	12	60	168	360	660	
$\Delta^2 f$	12	48	108	192	300		
$\Delta^3 f$	36	60	84	108			
$\Delta^4 f$	24	24	24				
$\Delta^5 f$	0	0					

The constant fourth difference equals 4! times the leading coefficient. This is a theorem for this polynomial: taking a forward difference lowers its degree by one and multiplies the leading coefficient by its degree. A constant row in a finite table for an unspecified sequence would only suggest a polynomial rule.

Exercise 3.2. Cube residues modulo 9

Hint. Keep the residue attached to its representative.

Solution. The cubes of $0, 1, \ldots, 8$ are congruent to $0, 1, 8, 0, 1, 8, 0, 1, 8$. Thus every cube is 0, 1, or -1 modulo 9. A sum of three cubes is congruent to one of $-3, -2, -1, 0, 1, 2, 3$. Consequently no integer congruent to 4 or 5 modulo 9 can be a sum of three integer cubes. This is a necessary obstruction; the residue argument does not assert that every other integer has such a representation.

Exercise 3.3. The first failure of $n^2 + n + 17$

Hint. Specify the search as $n = 0, 1, 2, \ldots$ and test primality exactly.

Solution. For $0 \le n \le 15$ the values are

$$17, 19, 23, 29, 37, 47, 59, 73, 89, 107, 127, 149, 173, 199, 227, 257,$$

and each is prime. At $n = 16$ the value is $289 = 17^2$. Trial division of the earlier values through their integer square roots certifies that 16 is the first failure in this nonnegative search. The construction explains it: at $n = p - 1$, the polynomial $n^2 + n + p$ equals p^2. If negative integers are permitted, "first" needs a different ordering convention.

Exercise 3.4. Two independent square tests

Hint. Compare an integer-root test with a search using only multiplication.

Solution. Both functions below accept nonnegative integers and reject other inputs.

```
from math import isqrt

def square_root_test(n):
    if type(n) is not int or n < 0:
        raise ValueError("expected integer >= 0")
    r = isqrt(n)
    return r*r == n

def square_search_test(n):
    if type(n) is not int or n < 0:
        raise ValueError("expected integer >= 0")
    left, right = 0, n + 1
    while left < right:
        mid = (left + right) // 2
        if mid*mid < n:
            left = mid + 1
        else:
            right = mid
    return left*left == n

assert all(square_root_test(n) == square_search_test(n)
           for n in range(10001))
```

The second search finds the least nonnegative integer with square at least n; equality is precisely the desired property. It avoids a floating-point square root, which can misclassify sufficiently large integers.

Exercise 3.5. Labelled graphs on seven vertices

Hint. An edge is chosen independently for every unordered pair.

Solution. There are $\binom{7}{2} = 21$ possible edges. A simple undirected graph on the fixed labelled vertex set is determined by a subset of these edges, so the exact count is

$$2^{21} = 2,097,152 \approx 2.1 \text{ million}.$$

This count includes disconnected graphs and isolated vertices. It does not identify graphs obtained from one another by relabelling.

Exercise 3.6. A conjecture log

Hint. Keep the failed statement instead of silently replacing it.

One model response. Question: does a prime exponent p always give a prime $2^p - 1$? Initial tests at $p = 2, 3, 5, 7$ succeed. The next test, $p = 11$, fails because $2^{11} - 1 = 2047 = 23 \cdot 89$. The corrected statement is the converse implication: if $2^n - 1$ is prime, then n is prime. The failed input, its factorization, the ordered list of tested exponents, and the corrected proof should all remain in the log. Thus a prime exponent is necessary but not sufficient. Exercise 3.8 proves the corrected statement.

Exercise 3.7. Mersenne numbers through exponent 31

Hint. Enumerate prime exponents and retain complete factorizations for composite outputs.
Solution.

p	$2^p - 1$	Conclusion or certificate
2	3	prime
3	7	prime
5	31	prime
7	127	prime
11	2,047	$23 \cdot 89$
13	8,191	prime
17	131,071	prime
19	524,287	prime
23	8,388,607	$47 \cdot 178481$
29	536,870,911	$233 \cdot 1103 \cdot 2089$
31	2,147,483,647	prime

A complete trial-division implementation is sufficient here.

```
from math import isqrt

def factors(n):
    if type(n) is not int or n < 2:
        raise ValueError("expected integer >= 2")
    result = []
    d = 2
    while d*d <= n:
        while n % d == 0:
            result.append(d)
            n //= d
        d += 1
    if n > 1:
        result.append(n)
    return result

prime_exponents = [p for p in range(2, 32)
                   if factors(p) == [p]]
for p in prime_exponents:
    n = 2**p - 1
    fs = factors(n)
    from math import prod
    assert prod(fs) == n
    assert all(factors(q) == [q] for q in fs)
```

When the loop ends, any remaining factor is prime: a composite remainder would have a prime divisor no greater than its square root. For the largest prime on the list, trial division needs no candidate beyond 46340. The search proves these individual primality conclusions; it does not settle larger exponents.

Exercise 3.8. When $a^n - 1$ can be prime

Hint. Factor once using a composite exponent, and once using $a - 1$.

Solution. If $n = uv$ with $u, v > 1$, then

$$a^n - 1 = (a^u - 1)(1 + a^u + \cdots + a^{u(v-1)}).$$

Both factors exceed one, contrary to primality. Thus n is prime. In fact, for every $n > 1$,

$$a^n - 1 = (a - 1)(1 + a + \cdots + a^{n-1}).$$

The second factor exceeds one, so primality forces $a - 1 = 1$, that is, $a = 2$. Neither restriction guarantees primality, as $2^{11} - 1$ shows.

Part I Capstone: A Worked Model

Hint. Choose an increasing search order so that a first failure is meaningful.

One model response. Claim: for every prime p, $2^p - 1$ is prime. Search prime exponents in increasing order through 31, using the factorization routine above. Include the boundary exponent 2, and check the first four results by hand. The smallest counterexample is $p = 11$, certified by $23 \cdot 89 = 2047$. An independent checker multiplies the factors and checks that both exceed one. To show that no smaller prime exponent fails, it also verifies the primality of the four earlier Mersenne values. Keep the table above, the code, and the search order. The corrected theorem and its proof are Exercise 3.8; no assertion about infinitely many Mersenne primes follows.

Chapter 4
From a Question to a Trustworthy Mathematical Answer

Mathematical Checkpoint

Hint. Validation establishes the domain; the nested loops then implement $i < j$ and $p_i > p_j$.

Solution. Using positions numbered from zero, $(2, 0, 1)$ has inversion pairs $(0, 1), (0, 2)$, and $(2, 1, 0)$ has all three pairs $(0, 1), (0, 2), (1, 2)$. An increasing permutation has zero inversions; a decreasing permutation has $\binom{n}{2} = n(n-1)/2$, because every pair is inverted. These check the count. The inputs $(0, 0)$ and $(-1, 0)$ fail the definition of a permutation of $0, 1$ and must be rejected before counting.

The type check excludes Booleans even though Python treats them as integer subclasses. Sorting a copy tests whether the entries are exactly $0, \ldots, n-1$ without modifying the original list.

```
def inversions(p):
    n = len(p)
    if any(type(x) is not int for x in p):
        raise ValueError("entries must be integers")
    if sorted(p) != list(range(n)):
        raise ValueError("expected permutation 0,...,n-1")
    count = 0
    for i in range(n):
        for j in range(i + 1, n):
            if p[i] > p[j]:
                count += 1
    return count

def inversions_compact(p):
    if any(type(x) is not int for x in p):
        raise ValueError("entries must be integers")
    if sorted(p) != list(range(len(p))):
        raise ValueError("expected permutation")
    return sum(p[i] > p[j] for i in range(len(p))
               for j in range(i + 1, len(p)))

for bad in [[0, 0], [-1, 0], [False]]:
    try:
        inversions(bad)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid input accepted")
p = [2, 0, 3, 1]
```

```
before = p.copy()
assert inversions(p) == 3 and p == before
from itertools import permutations
for n in range(8):
    for p in permutations(range(n)):
        assert inversions(p) == inversions_compact(p)
assert inversions((2, 0, 1)) == 2
assert inversions((2, 1, 0)) == 3
for n in range(11):
    assert inversions(tuple(range(n))) == 0
    assert inversions(tuple(reversed(range(n)))) == n*(n-1)//2
```

The sum counts true comparisons as ones. Both implementations return zero on the empty permutation. Their comparison exhausts sizes 0 through 7; the loop description explains their agreement in general.

Exercise 4.1. A six-part specification

Hint. Make every convention needed to interpret the output explicit.

One model response. Replace “find divisors” with the following specification.

1. Objective: return all positive divisors of an integer.
2. Definitions: d divides n when $n \bmod d = 0$; include 1 and n.
3. Inputs: a Python integer $n \geq 1$, excluding Booleans; otherwise raise `ValueError`.
4. Output: a strictly increasing list, with no repetitions; do not print inside the function.
5. Constraints: exact integer arithmetic, standard library only, candidate divisors at most $\lfloor \sqrt{n} \rfloor$.
6. Checks: $1 \mapsto [1]$, $9 \mapsto [1, 3, 9]$, $12 \mapsto [1, 2, 3, 4, 6, 12]$; compare with direct enumeration for $1 \leq n \leq 10000$ and verify invalid-input rejection.

The specification says both what to calculate and how the program should accept inputs and return the answer.

Exercise 4.2. A square and an invalid input

Hint. A strict square-root bound omits the middle divisor of a square.

Solution. Predict $d(9) = 3$ and rejection of zero before running the experiment. The book’s faulty loop misses the divisor 3 at $n = 9$, returning 2 instead of 3. Add the regression test $d(9) = 3$, and require rejection of zero. The case $n = 1$ is another valuable boundary square.

```
from math import isqrt

def divisor_count(n):
    if type(n) is not int or n < 1:
        raise ValueError("expected positive integer")
    count = 0
    for d in range(1, isqrt(n) + 1):
        if n % d == 0:
            count += 1 if d*d == n else 2
    return count
```

```
assert divisor_count(9) == 3
assert divisor_count(1) == 1
for bad in [0, -1, 3.5, True]:
    try:
        divisor_count(bad)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid input accepted")
```

The inclusive endpoint finds a square's middle divisor; adding one in that case prevents counting it twice.

Exercise 4.3. Describe a mathematical experiment

Hint. Identify the objects counted before discussing how the calculation runs.

One model response. Consider the divisor-count experiment at $n = 12$. Its input is a positive integer. The counted objects are positive integers d satisfying $d \mid 12$, namely $1, 2, 3, 4, 6, 12$; the answer is six. Direct division checks every candidate from 1 to 12. Independently, the factorization $12 = 2^2 \cdot 3$ gives $(2 + 1)(1 + 1) = 6$ divisors. The square case $n = 9$ has three divisors, with the middle divisor counted once; $n = 1$ has one. Zero is outside the stated domain. This describes the input, what we count, the answer, and how we check it, without needing to discuss the Python.

Exercise 4.4. A certificate and an independent checker

Hint. A witness should be much easier to check than to discover.

One model response. For the factorization answer $536870911 = 233 \cdot 1103 \cdot 2089$, the certificate is the three-factor list. Multiplication verifies the identity and confirms compositeness when at least two factors exceed one. To certify a complete prime factorization, also test every factor for primality. These are different claims: a product identity alone need not certify that its factors are prime.

For a path, use the vertex sequence and check endpoints and consecutive edges. For a claimed linear dependence among columns of A, use a nonzero coefficient vector c and verify exactly that $Ac = 0$. A zero vector is never a dependence certificate.

Exercise 4.5. Transparent and optimized implementations

Hint. Compare against the definition on every input in a declared range.

Solution. The direct implementation counts every divisor candidate.

```
def divisor_count_reference(n):
    if type(n) is not int or n < 1:
        raise ValueError("expected positive integer")
    return sum(n % d == 0 for d in range(1, n + 1))

assert all(divisor_count(n) == divisor_count_reference(n)
           for n in range(1, 10001))
```

The comparison passes for all $10,000$ inputs. The reference uses n divisibility tests; the paired version uses $\lfloor \sqrt{n} \rfloor$. The range includes many squares, primes, and composites and is small enough for complete comparison. This comparison checks the two implementations on that range. The pairing argument in Chapter 2 proves correctness beyond that range.

Further Investigation: A Worked Model

Hint. Use a computation with a certificate available from elementary algebra.

One model response. Build a gcd function for nonnegative integers. Reject $(0, 0)$ if the chosen contract leaves that case undefined; accept $(0, b)$ for $b > 0$. Retain $(391, 299)$ with answer 23 and Bezout coefficients $(-3, 4)$. A reference checks common divisors directly on positive inputs through 100. The function under test uses the Euclidean algorithm; an independent certificate checker tests divisibility and the Bezout identity. The invariant and termination proof in Exercise 11.1 justify the unrestricted algorithm.

Chapter 5
Exact and Approximate Methods

Mathematical Checkpoint

Hint. A rounded multiplication may hide the error already present in an input.
Solution.

```
from fractions import Fraction
import math
exact = Fraction(1, 3)
approx = 1 / 3
assert 3 * exact == 1
assert 3 * approx == 1.0
assert Fraction(approx) != exact
assert Fraction(approx) - exact == -Fraction(1, 54043195528445952)
assert math.isclose(approx, float(exact),
                    rel_tol=1e-12, abs_tol=1e-15)
```

The stored float is 6004799503160661/18014398509481984. Its multiplication by three rounds to 1.0; that equality does not make the original stored value exactly 1/3. The tolerance comparison above asks whether two floating-point approximations are acceptably close. Here they are identical after conversion, so it is not an independent check of accuracy. Comparing the exact fractions exposes the input error.

Near zero, compare 0 and 10^{-14}. Their absolute difference is 10^{-14}, within a chosen absolute bound 10^{-12}. With only a relative tolerance of 10^{-6}, the permitted difference is $10^{-6}\max(0, 10^{-14}) = 10^{-20}$, so the comparison fails. A relative error measured against the exact value zero is undefined. The useful absolute bound must come from the scale and purpose of the calculation.

```
assert not math.isclose(0.0, 1e-14, rel_tol=1e-6, abs_tol=0.0)
assert math.isclose(0.0, 1e-14, rel_tol=1e-6, abs_tol=1e-12)
```

Exercise 5.1. Equal expressions, different domains

Hint. Cancelling a denominator does not restore its excluded zeros.
Solution. Over the reals,

$$\frac{x^2-1}{x-1} = x+1 \qquad (x \neq 1).$$

The fraction has natural domain $\mathbb{R} \setminus \{1\}$; the polynomial has domain $\mathbb{R}$. A second example is $x/x = 1$ for $x \neq 0$, with domains $\mathbb{R} \setminus \{0\}$ and $\mathbb{R}$. In each case the expressions agree on their common domain, but the original expression remains undefined at the excluded value. Simplification must preserve that restriction.

Exercise 5.2. Verify a symbolic solution

Hint. Substitute into the original equation, including its domain conditions.

Solution. Solve $\sqrt{x+2} = x$ over the reals. Squaring gives $x + 2 = x^2$, hence candidates 2 and -1. The original right side must be nonnegative. Substitution gives $\sqrt{4} = 2$ for $x = 2$, whereas $\sqrt{1} \neq -1$. Thus the solution set is $\{2\}$.

```
import sympy as sp
x = sp.symbols('x', real=True)
candidates = sp.solve(x*x - x - 2, x)
solutions = [c for c in candidates
             if sp.simplify(sp.sqrt(c + 2) - c) == 0]
assert solutions == [2]
```

Squaring gives a necessary condition but loses the sign restriction. Substitution in the original equation rules out the extraneous candidate.

Exercise 5.3. Cancellation in $\sqrt{1+x} - 1$

Hint. Multiply numerator and denominator by $\sqrt{1+x} + 1$.

Solution. For $x \geq -1$,

$$\sqrt{1+x} - 1 = \frac{x}{\sqrt{1+x}+1}.$$

The denominator is positive on this domain. Near zero the second expression avoids subtracting two nearly equal numbers.

x	Direct	Rationalized
1×10^{-4}	4.999875×10^{-5}	4.999875×10^{-5}
1×10^{-8}	5.000000×10^{-9}	5.000000×10^{-9}
1×10^{-12}	5.000445×10^{-13}	5.000000×10^{-13}
1×10^{-16}	0	5.000000×10^{-17}
1×10^{-20}	0	5.000000×10^{-21}

At $x = 10^{-16}$, the stored sum $1 + x$ rounds to 1, so the direct result is zero. The stable result is $5 \cdot 10^{-17}$, consistent with $\sqrt{1+x} - 1 = x/2 + O(x^2)$.

```
import math
for x in [1e-4, 1e-8, 1e-12, 1e-16, 1e-20]:
    direct = math.sqrt(1 + x) - 1
    stable = x / (math.sqrt(1 + x) + 1)
    assert stable > 0
```

The stable formula removes this cancellation; it does not make every floating-point operation exact.

Exercise 5.4. Absolute and relative tolerances

Hint. Near zero use an absolute scale; for large quantities allow an error proportional to their size.

One model response. Suppose errors below 10^{-12} in a dimensionless near-zero quantity are immaterial. Use `abs_tol=1e-12` and `rel_tol=0`. Thus $7 \cdot 10^{-13}$ is close to zero and $2 \cdot 10^{-12}$ is not. Near 10^6, suppose relative error 10^{-9} is acceptable; this allows approximately 10^{-3} absolute error. Use `rel_tol=1e-9`, with a small absolute tolerance for exceptional values near zero.

```
assert math.isclose(7e-13, 0.0, rel_tol=0, abs_tol=1e-12)
assert not math.isclose(2e-12, 0.0, rel_tol=0, abs_tol=1e-12)
assert math.isclose(1e6 + 0.0005, 1e6,
                    rel_tol=1e-9, abs_tol=1e-12)
assert not math.isclose(1e6 + 0.002, 1e6,
                        rel_tol=1e-9, abs_tol=1e-12)
```

These tolerances specify the error we are prepared to accept. Their suitability depends on the units, measurement uncertainty, and how we will use the result.

Exercise 5.5. A bracket or residual certificate

Hint. A sign-changing interval needs continuity and reliable signs.

Solution. For $f(x) = x^2 - 2$, take the exact rational bracket

$$a = 1414213/10^6, \qquad b = 1414214/10^6.$$

Here

$$a^2 - 2 = -1590631/10^{12} < 0, \qquad b^2 - 2 = 1237796/10^{12} > 0.$$

Continuity gives a root in $[a, b]$. Strict increase on this positive interval gives uniqueness. The midpoint $2828427/2000000$ therefore approximates $\sqrt{2}$ within $5 \cdot 10^{-7}$.

A residual $|q^2 - 2|$ checks the equation at an approximate answer q. For $q \geq 1$, factorization gives $|q - \sqrt{2}| = |q^2 - 2|/(q + \sqrt{2}) \leq |q^2 - 2|/2$. This error bound requires a reliable bound on the true residual. An arbitrary small floating-point residual alone is not a rigorous forward-error certificate, particularly for a poorly conditioned equation.

Exercise 5.6. Stable roots of $x^2 + Bx + 1 = 0$

Hint. The roots have product one; first compute the root with larger magnitude.

Solution. For $B \geq 2$, both roots are negative. Compute

$$x_{\text{large}} = -\frac{B}{2}\left(1 + \sqrt{(1 - 2/B)(1 + 2/B)}\right), \qquad x_{\text{small}} = \frac{1}{x_{\text{large}}}.$$

The sum inside the parentheses has no cancellation, and scaling avoids forming B^2. At $B = 2$ both roots are -1.

```
def negative_quadratic_roots(B):
    if not math.isfinite(B) or B < 2:
        raise ValueError("finite B >= 2 required")
    t = 2.0 / B
    s = math.sqrt((1 - t) * (1 + t))
    large = -(B / 2) * (1 + s)
    direct = (-B + B*s) / 2
    stable = 1 / large
    return large, direct, stable

assert negative_quadratic_roots(2) == (-1, -1, -1)
for B in [1e4, 1e8, 1e12, 1e200]:
    large, direct, small = negative_quadratic_roots(B)
    assert large < 0 and small < 0
    assert math.isclose(large*small, 1, rel_tol=1e-14)
```

For a real coefficient with $B \leq -2$, reverse the sign of the first root: more generally use $-(B/2)(1+s)$ with $s = \sqrt{(1 - 2/|B|)(1 + 2/|B|)}$. For $|B| < 2$ the roots are nonreal, so this real-valued routine does not cover that case.

Exercise 5.7. The first zero direct small root

Hint. Reproduce the exact arithmetic expression; algebraically equivalent implementations can round differently.

Solution. For the preceding scaled implementation the magnitudes are:

B	Direct small-root magnitude	Stable magnitude
10^4	1.000000×10^{-4}	1.000000×10^{-4}
10^6	9.999494×10^{-7}	1.000000×10^{-6}
10^7	1.005828×10^{-7}	1.000000×10^{-7}
10^8	7.450581×10^{-9}	1.000000×10^{-8}
10^9	5.960464×10^{-8}	1.000000×10^{-9}
10^{10}	0	1.000000×10^{-10}
10^{12}	0	1.000000×10^{-12}
10^{16}	0	1.000000×10^{-16}

Among the tested powers $B = 10^k$, $1 \le k \le 16$, the first zero occurs at $B = 10^{10}$ in the recorded environment. The negative-coefficient version in the book has the same magnitudes. The surprising nonzero value at 10^9 is inaccurate: the direct expression is about sixty times the stable result. Cancellation error need not grow monotonically row by row.

```
scan = [(k, negative_quadratic_roots(10.0**k))
        for k in range(1, 17)]
first_zero = next(k for k, roots in scan if roots[1] == 0)
assert first_zero == 10  # recorded binary64 environment
```

The assertion records this run; a different evaluation order or platform should report its own scan. In particular, forming $\sqrt{B^2 - 4}$ directly is a different implementation and can have a different first-zero threshold. In all these cases, the true small root remains nonzero.

Further Investigation: A Worked Model

Hint. Use exact decimal input when comparing higher precisions.

One model response. Take $x = 10^{-20}$ and $g(x) = \sqrt{1 + x} - 1$. Its exact symbolic form is $\sqrt{1 + 10^{-20}} - 1$, approximately $5 \cdot 10^{-21} - 1.25 \cdot 10^{-41}$. Binary64 direct evaluation gives zero; rationalization gives approximately $5 \cdot 10^{-21}$. Decimal arithmetic with 30 and 60 digits resolves the subtraction; creating `Decimal("1e-20")` preserves the intended input.

```
from decimal import Decimal, localcontext
for precision in [30, 60]:
    with localcontext() as ctx:
        ctx.prec = precision
        x = Decimal("1e-20")
        direct = (1 + x).sqrt() - 1
        stable = x / ((1 + x).sqrt() + 1)
        assert direct > 0 and stable > 0
```

The identity $g(x)(g(x) + 2) = x$ supplies a separate equation to check. The first omitted term above explains the scale of the approximation; increasing precision supplies numerical evidence, not exact equality of stored decimals with the irrational result.

Chapter 6
Pictures, Tables, and Evidence

Mathematical Checkpoint

Hint. Check the expected columns and field types before converting the rows and checking their values.

Solution. A correct table includes $(2, 4), (3, 9)$. Altering its last entry to $(3, 8)$ gives a mathematically false table that can still be saved and reloaded without any change. Preservation and correctness are separate questions. Checking each second entry against the square of the first detects $8 \neq 3^2$.

The book's square-data CSV has columns `n` and `square`. The following reader raises an error on a bad header, malformed row, or incorrect square.

```
import csv
from io import StringIO

def read_squares(stream):
    reader = csv.DictReader(stream)
    if reader.fieldnames != ["n", "square"]:
        raise ValueError("expected columns n,square")
    rows = []
    for number, row in enumerate(reader, start=2):
        if set(row) != {"n", "square"}:
            raise ValueError(f"bad fields on line {number}")
        try:
            n, square = int(row["n"]), int(row["square"])
        except (TypeError, ValueError) as exc:
            raise ValueError(f"bad integer on line {number}") from exc
        if square != n*n:
            raise ValueError(f"wrong square on line {number}")
        rows.append((n, square))
    return rows

assert read_squares(StringIO("n,square\n2,4\n3,9\n")) == [(2,4),(3,9)]
for bad in ["n,square\n3,8\n", "n,value\n3,9\n"]:
    try:
        read_squares(StringIO(bad))
    except ValueError:
        pass
    else:
        raise AssertionError("invalid CSV accepted")
def round_trip(rows):
    stream = StringIO(newline="")
```

```
    writer = csv.writer(stream)
    writer.writerow(["n", "square"])
    writer.writerows(rows)
    stream.seek(0)
    reader = csv.DictReader(stream)
    return [(int(r["n"]), int(r["square"])) for r in reader]

for original in [[(2, 4), (3, 9)], [(2, 4), (3, 8)]]:
    assert round_trip(original) == original
assert any(square != n*n for n, square in round_trip([(2, 4), (3, 8)]))
```

A round trip proves that writing and reading preserved the recorded values. A wrong formula could survive such a round trip perfectly. The comparison with n^2 is the independent mathematical check.

Exercise 6.1. The question behind a figure

Hint. State a relationship that the viewer is supposed to examine.

One model response. The convergence figure asks whether $E_n = e - (1 + 1/n)^n$ appears to decrease approximately as a constant times $1/n$. Plotting E_n against n, alongside $e/(2n)$, makes that question visible. Plotting the scaled quantity nE_n asks more specifically whether it tends toward $e/2$. Neither finite display by itself proves an asymptotic formula.

Exercise 6.2. Linear and logarithmic scales

Hint. Use exactly the same rows in both panels.

Solution. Use the 200 errors in Exercise 6.6. Linear axes give equal visual space to equal absolute changes and emphasize the first few large errors. Log–log axes give equal visual space to equal multiplicative changes; a power law $E_n \approx Cn^{-1}$ has slope -1 there. Both displays represent the same data. A semilogarithmic plot would be a different test, suited to an exponential law. Logarithmic axes cannot display zero or negative errors without an explicit convention.

Exercise 6.3. A table, summary, and figure from one experiment

Hint. Return rows before deciding how to draw them.

Solution.

```
import math
import matplotlib.pyplot as plt

def convergence_data(ns):
    rows = []
    for n in ns:
        if type(n) is not int or n < 1:
            raise ValueError("positive integer n required")
        value = math.exp(n * math.log1p(1 / n))
```

```
        error = abs(math.e - value)
        rows.append({"n": n, "value": value,
                     "error": error, "scaled": n*error})
    return rows

def summarize(rows):
    if not rows:
        raise ValueError("nonempty data required")
    return {"count": len(rows),
            "largest_error": max(r["error"] for r in rows),
            "last_scaled_error": rows[-1]["scaled"]}

def plot_errors(rows):
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
    ns = [r["n"] for r in rows]
    errors = [r["error"] for r in rows]
    if any(e <= 0 for e in errors):
        raise ValueError("log plot requires positive errors")
    for ax in axes:
        ax.plot(ns, errors, ".", markersize=2)
        ax.plot(ns, [math.e/(2*n) for n in ns], "--")
        ax.set_xlabel("n")
        ax.set_ylabel("Absolute error")
    axes[1].set_xscale("log")
    axes[1].set_yscale("log")
    fig.tight_layout()
    return fig

rows = convergence_data(range(1, 201))
assert summarize(rows)["count"] == 200
print(summarize(rows))
for row in rows:
    if row["n"] in [1, 2, 10, 50, 100, 200]:
        print(row)
fig = plot_errors(rows)
plt.close(fig)
```

The full data set has one row for each $n = 1, \ldots, 200$. Selected rows are shown in Exercise 6.6, and the following figure uses all 200 rows. The summary is: 200 rows, largest absolute error 0.718281828459 at $n = 1$, and last scaled error 1.352941105934 at $n = 200$. The two panels have the same input and error coordinates; only their axes change. The row at $n = 200$, multiplied by 200, agrees with the summary. The data can also be saved as CSV or used in another calculation. Keep the exact function and the arithmetic mode as well, since the word “error” here uses the stored value of `math.e`.

Exercise 6.4. A reproduction note

Hint. A seed is necessary for a seeded experiment but is not its whole description.

One model response. For the walk in Chapter 8, retain: simple symmetric walk on $\mathbb{Z}$; initial position 0; 40 steps; increments selected by `random.Random(17).choice([-1, 1])`; seed 17; complete path retained; endpoint, minimum, and maximum include the initial position; interpreter version and script version recorded. State that one seeded path is one reproducible sample, not an estimate of

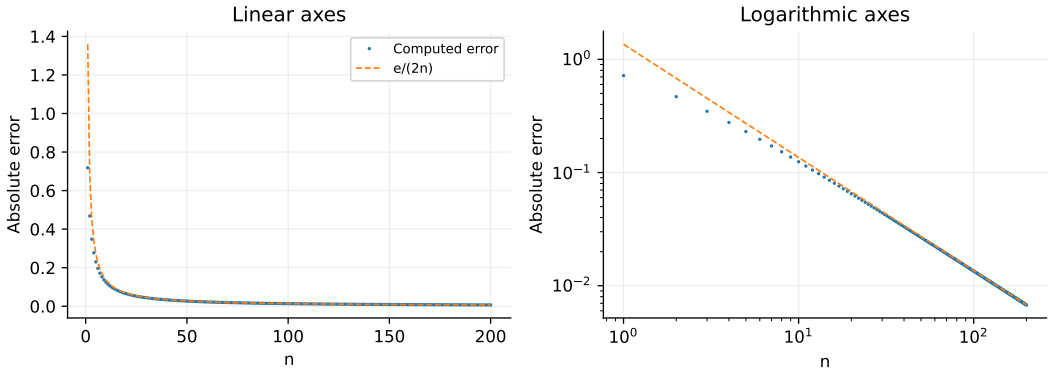


Fig. 6.1 The same 200 errors on linear and logarithmic axes, with $e/(2n)$ for comparison. The selected table and summary describe these data.

the entire distribution. For an approximate calculation also record precision, formula, tolerance, and the meaning of each error estimate.

Exercise 6.5. A stopping rule

Hint. Define completion in terms of the mathematical question and its checks.

One model response. A divisor-list experiment is complete when it meets the specification in Exercise 4.1, passes the boundary and complete-range checks, returns sorted certified divisors, and has a reproducible example. Stop adding features when these requirements are met. Consider a faster algorithm if the intended input sizes make the current one impractical. For a search run, declare a finite upper bound or time budget in advance; if a time budget expires, record the last fully completed input and mark the rest unsearched.

Exercise 6.6. The convergence figure through $n = 200$

Hint. Change the sample locations separately from the axis scales.

Solution. Generate every integer $1 \le n \le 200$, then draw both panels. Figure 6.1 in Exercise 6.3 displays these two panels.

n	E_n (careful evaluation)	nE_n
1	0.718281828459	0.718281828
2	0.468281828459	0.936563657
10	0.124539368359	1.245393684
50	0.026693799385	1.334689969
100	0.013467999038	1.346799904
200	0.006764705530	1.352941106

The last scaled error is approximately 1.352941106, approaching $e/2 \approx 1.359140914$. Equally spaced integers devote most observations to the larger n region; sampling $1, 2, 4, \ldots$ devotes roughly equal numbers of observations to successive multiplicative ranges. A logarithmic axis changes the display, while logarithmic sampling changes which rows are computed. These choices are independent.

Exercise 6.7. A more careful evaluation

Hint. Compute $\exp(n \log(1 + 1/n))$ using `log1p`.

Solution. Replace `(1.0 + 1.0/n)**n` by `math.exp(n * math.log1p(1.0/n))`. For $1 \le n \le 200$, the largest relative change in the computed error is about $8.54 \cdot 10^{-12}$ in this run. If "material" means a change exceeding one part in 10^6 of the error, no row changes materially; the plotted interpretation is unchanged.

Larger n exposes the reason for the repair. At $n = 10^{12}$ the ordinary formula gives approximately 2.71852349603724, even exceeding e; the careful formula gives 2.71828182845769. At $n = 10^{16}$ the ordinary formula gives 1, whereas the careful formula rounds to the stored e. The true error there is still approximately $1.35914 \cdot 10^{-16}$, so the last subtraction also needs higher precision to resolve the error itself. The improved expression avoids the earlier loss of information, but cancellation can still occur in the final subtraction.

Part II Capstone: A Reusable Divisor Module

Hint. A complete divisor list itself supplies witnesses for divisibility; completeness needs an argument as well.

One model response. Use the paired divisor function in Appendix A, Exercise A.4, with its explicit positive-integer contract. Retain the direct reference in Exercise 4.5, the exact range $1 \le n \le 10000$, and the invalid-input checks. Return a dictionary containing n, its ordered divisor list, and its count. A display function produces the ten-row table from Exercise 2.1. Each reported divisor can be checked by remainder, and the paired-divisor proof certifies completeness. The paired routine and its reference use only the standard library; no seed or approximation is involved. The supplied `programs/divisor_tool.py` implements this complete model, including its table and fast check. The stopping rule is the one in Exercise 6.5.

Chapter 7
Sequences and Linear Algebra

Mathematical Checkpoint

Hint. Save both old state values before forming the new state.

Solution. Starting with $2, 5$, the next values are $3 \cdot 5 - 2 \cdot 2 = 11$, $3 \cdot 11 - 2 \cdot 5 = 23$, $3 \cdot 23 - 2 \cdot 11 = 47$, and $3 \cdot 47 - 2 \cdot 23 = 95$. The three requested lists are [], [2], and $[2, 5, 11, 23, 47, 95]$. For $b_n = 3 \cdot 2^n - 1$, we have $b_0 = 2, b_1 = 5$ and

$$3b_{n+1} - 2b_n = 9 \cdot 2^{n+1} - 6 \cdot 2^n - 1 = 3 \cdot 2^{n+2} - 1 = b_{n+2}.$$

The recurrence and initial values determine every term, so induction proves $a_n = b_n$.

The chapter's recurrence has $a_0 = 2, a_1 = 5$ and $a_{n+2} = 3a_{n+1} - 2a_n$.

```
def recurrence_terms(count):
    if type(count) is not int or count < 0:
        raise ValueError("expected integer >= 0")
    a, b = 2, 5
    terms = []
    for _ in range(count):
        terms.append(a)
        a, b = b, 3*b - 2*a
    return terms

assert recurrence_terms(0) == []
assert recurrence_terms(1) == [2]
assert recurrence_terms(6) == [2, 5, 11, 23, 47, 95]
assert recurrence_terms(6) == [3*2**n - 1 for n in range(6)]
```

Both expressions on the right of simultaneous assignment are evaluated from the old state. Two sequential assignments using an already changed `a` would implement a different recurrence. The closed form $3 \cdot 2^n - 1$ has the required initial values and satisfies the recurrence, so induction proves it for every $n \geq 0$.

Exercise 7.1. Solve $a_{n+2} = 5a_{n+1} - 6a_n$

Hint. Factor $t^2 - 5t + 6$.

Solution. The characteristic roots are 2 and 3. Write $a_n = A2^n + B3^n$. The initial conditions give $A + B = 1$ and $2A + 3B = 4$, so $A = -1, B = 2$. Therefore

$$a_n = 2 \cdot 3^n - 2^n.$$

Its first six terms are $1, 4, 14, 46, 146, 454$. Both 2^n and 3^n satisfy the recurrence, so their indicated linear combination does as well. The initial conditions then prove uniqueness by induction.

```
a = [1, 4]
while len(a) < 15:
    a.append(5*a[-1] - 6*a[-2])
assert a == [2*3**n - 2**n for n in range(15)]
```

Exercise 7.2. Fibonacci periods

Hint. Detect return of the ordered state pair $(0, 1)$, not merely a zero term.

Solution. Generate consecutive residue pairs, stopping at the first positive return to $(0, 1)$. The calculation below reconstructs the periods modulo $2, 3, 5, 10$ as $3, 8, 20, 60$, without using the printed table as input. Modulo 2, the complete cycle is

$$(0, 1) \longmapsto (1, 1) \longmapsto (1, 0) \longmapsto (0, 1).$$

The two intermediate pairs differ from $(0, 1)$, so the initial pair returns after exactly three steps and not earlier. A return of the pair repeats the entire future sequence, because the update is deterministic; any period of the sequence must return both initial residues. Therefore the least period modulo 2 is 3.

```
def fibonacci_period(m):
    if type(m) is not int or m <= 0:
        raise ValueError("positive integer modulus required")
    a, b = 0, 1 % m
    initial = (a, b)
    for period in range(1, m*m + 1):
        a, b = b, (a+b) % m
        if (a, b) == initial:
            return period
    raise AssertionError("state must return")

periods = {m: fibonacci_period(m) for m in (2, 3, 5, 10)}
print(periods)
assert periods == {2: 3, 3: 8, 5: 20, 10: 60}

states = [(0, 1)]
for _ in range(periods[2]):
    a, b = states[-1]
    states.append((b, (a+b) % 2))
assert states == [(0, 1), (1, 1), (1, 0), (0, 1)]
assert states[0] not in states[1:-1]
```

The update $(a, b) \mapsto (b, a + b)$ is invertible modulo m, with inverse $(c, d) \mapsto (d - c, c)$. On the finite set of m^2 pairs, the orbit is periodic from the start. The first positive return gives the minimal

period. Since congruence modulo 10 is equivalent to congruence modulo both 2 and 5, the last period is also $\operatorname{lcm}(3, 20) = 60$.

Exercise 7.3. A generating function and fifteen coefficients

Hint. Multiply the recurrence by x^{n+2} and sum for $n \geq 0$.

Solution. For Exercise 7.1, let $A(x) = \sum_{n\geq 0} a_n x^n$. Then $A - 1 - 4x = 5x(A - 1) - 6x^2 A$, whence

$$A(x) = \frac{1 - x}{1 - 5x + 6x^2} = \frac{2}{1 - 3x} - \frac{1}{1 - 2x}.$$

The coefficients for $n = 0, \ldots, 14$ are

$$1, 4, 14, 46, 146, 454, 1394, 4246, 12866, 38854,$$
$$117074, 352246, 1058786, 3180454, 9549554.$$

```
import sympy as sp
x = sp.symbols('x')
A = (1-x) / (1-5*x+6*x*x)
series = sp.series(A, x, 0, 15).removeO().expand()
coefficients = [series.coeff(x, n) for n in range(15)]
assert coefficients == [2*3**n - 2**n for n in range(15)]
```

This is an identity of formal power series; analytic convergence need not be assumed to obtain the coefficient formula.

Exercise 7.4. A prefix fitting two recurrences

Hint. A longer-order recurrence may place no constraint on the earliest terms.

Solution. The prefix $1, 2, 4$ fits $a_{n+1} = 2a_n$ with $a_0 = 1$. It also fits $b_{n+3} = b_{n+2} + b_{n+1} + b_n$ with initial terms $1, 2, 4$. The next terms are 8 and 7, respectively. A withheld fourth term therefore distinguishes these two candidates. No fixed finite prefix uniquely determines an unrestricted recurrence: one can always choose a higher-order rule or a later change. Specify the class of recurrences you are considering, then try to prove the proposed rule from the sequence's definition.

Exercise 7.5. Rank over two fields

Hint. Choose an integer determinant divisible by 2 but nonzero.

Solution. Take

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}.$$

Its determinant is 2, so its rank over $\mathbb{Q}$ is two. Modulo 2 the matrix becomes $\left(\begin{smallmatrix} 1 & 0 \\ 0 & 0 \end{smallmatrix}\right)$, whose rank over $\mathbb{F}_2$ is one. The field is part of the question, not a numerical precision setting.

Exercise 7.6. An underdetermined rational system

Hint. Exhibit a particular solution plus a basis for the homogeneous solutions.

Solution. Consider

$$x + y + z = 1, \qquad 2x + 2y + 2z = 2.$$

Every rational solution is

$$(x, y, z) = (1, 0, 0) + s(-1, 1, 0) + t(-1, 0, 1), \qquad s, t \in \mathbb{Q}.$$

Substitution gives 1 in the first equation and 2 in the second. Conversely any solution has $x = 1-y-z$, so setting $s = y, t = z$ recovers it. The two homogeneous vectors are independent, as their second and third coordinates show. Thus the displayed family gives every rational solution.

Exercise 7.7. Rank with a declared absolute tolerance

Hint. Numerical rank counts singular values strictly above the chosen threshold.

Solution. Use $A_\varepsilon = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1+\varepsilon \end{smallmatrix}\right)$. Choose the absolute threshold $\tau = 10^{-10}$ and report $\#\{i : s_i > \tau\}$. For comparison also use 10^{-14}.

ε	Computed $s_{\min}$	$\tau = 10^{-10}$	$\tau = 10^{-14}$
1×10^{-8}	5.000×10^{-9}	2	2
1×10^{-12}	5.001×10^{-13}	1	2
1×10^{-15}	6.214×10^{-16}	1	1
1×10^{-16}	3.355×10^{-17}	1	1

For positive ε, the smaller singular value is approximately $\varepsilon/2$, while the larger is approximately 2. Thus 10^{-12} is unresolved at 10^{-10} but resolved at 10^{-14}. The 10^{-15} perturbation is stored, but its smaller singular value is below either chosen threshold. At 10^{-16}, $1+\varepsilon$ rounds to 1, so the stored matrix has identical rows. Its tiny computed second singular value is numerical roundoff. Over $\mathbb{Q}$, the intended matrix has determinant ε and rank two whenever $\varepsilon \neq 0$.

```
import numpy as np
for epsilon in [1e-8, 1e-12, 1e-15, 1e-16]:
    matrix = np.array([[1., 1.], [1., 1. + epsilon]])
    singular = np.linalg.svd(matrix, compute_uv=False)
    ranks = [int(np.count_nonzero(singular > tol))
             for tol in [1e-10, 1e-14]]
```

An absolute threshold must be interpreted in the units of the matrix; rescaling the matrix can change this classification.

Exercise 7.8. The change-of-basis convention

Hint. Test the two coordinate unit vectors; they determine a linear map.

Solution. In the book,

$$T = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \quad P = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad C = P^{-1}TP = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

For $e_1 = (1, 0)^T$, $T(Pe_1) = (1, 1)^T = P(Ce_1)$. For $e_2 = (0, 1)^T$, $T(Pe_2) = (2, 1)^T = P(Ce_2)$. Direct multiplication gives $TP = PC = \left(\begin{smallmatrix} 1 & 2 \\ 1 & 1 \end{smallmatrix}\right)$. Thus the two routes agree for every coordinate vector, since e_1, e_2 form a basis. The columns of P are the new basis in old coordinates: P converts new coordinates to old, and P^{-1} converts back.

Further Investigation: A Worked Model

Hint. Use the recurrence of Exercise 7.1 as an exact reference.

One model response. For $v_n = (a_{n+1}, a_n)^T$, use $M = \left(\begin{smallmatrix} 5 & -6 \\ 1 & 0 \end{smallmatrix}\right)$ and $v_0 = (4, 1)^T$. Then $v_n = M^n v_0$. Compute terms 0 through 9 to form a conjecture, and reserve 10 through 14 for verification against $2 \cdot 3^n - 2^n$. Reducing these integer results modulo 5 agrees with iterating the reduced matrix; reduction respects addition and multiplication. Floating-point iteration may introduce rounding as terms grow, so use the exact formula as a reference. The substitution proof establishes the closed form independently of all three computations.

Chapter 8
Dynamics and Randomness

Mathematical Checkpoint

Hint. Structural properties hold for every permitted random outcome.

Solution. If r of the n steps go right, the endpoint is $r - (n - r) = 2r - n$. Since $0 \le r \le n$, it lies in $[-n, n]$ and has the parity of n. For $n = 0$ the path is $[0]$. These conclusions hold for every permitted sequence of steps, regardless of how it was chosen. A distributional claim also needs a probability model for those choices.

```
import random

def random_walk(steps, seed):
    if type(steps) is not int or steps < 0:
        raise ValueError("nonnegative integer steps required")
    rng = random.Random(seed)
    path = [0]
    for _ in range(steps):
        path.append(path[-1] + rng.choice([-1, 1]))
    return path

def check_walk(path, steps):
    return (len(path) == steps + 1 and path[0] == 0
            and all(abs(b-a) == 1 for a, b in zip(path, path[1:]))
            and (path[-1] - steps) % 2 == 0
            and -steps <= path[-1] <= steps)

for n in [0, 1, 40, 100]:
    assert check_walk(random_walk(n, 17), n)
assert random_walk(0, 17) == [0]
assert random_walk(40, 17) == random_walk(100, 17)[:41]
```

There are $n + 1$ positions because the initial position is retained. Each step changes parity, so the endpoint has the parity of n. A fixed seed and unchanged sampling procedure give the prefix property. These checks would also pass for a walk that always steps right; they do not establish unbiased sampling or the endpoint distribution.

Exercise 8.1. The 40-step walk

Hint. Keep the step-choice list in the same order as the book.

Solution. With `choice([-1, 1])` and seed 17, the complete path is

$$0, 1, 2, 3, 4, 3, 4, 3, 2, 1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1, 0,$$
$$1, 0, -1, 0, 1, 0, 1, 0, 1, 2, 1, 2, 3, 2, 3, 2, 3, 4, 3, 4.$$

It has endpoint 4, minimum -1, and maximum 6, including the initial position. For seed 18 the corresponding summary is given in the retained results and the table below.

Seed	Endpoint	Minimum	Maximum
17	4	-1	6
18	6	-2	6

Changing the seed changes the realized path and usually these three statistics. It leaves the permitted increments, number of positions, endpoint parity, and the mathematical model unchanged. Under independent fair steps, the theoretical endpoint mean is 0 and variance is 40; these describe the distribution over all such walks, not any one path.

Exercise 8.2. Endpoint standard deviations

Hint. Use many complete walks and keep the same list of seeds at each length.

Solution. Use the fixed seed list $0, 1, \ldots, 999$. Compute the usual sample standard deviation, with divisor 999.

```
import statistics
seeds = list(range(1000))
rows = []
for n in [100, 400, 1600]:
    endpoints = [random_walk(n, seed)[-1] for seed in seeds]
    rows.append((n, statistics.mean(endpoints),
                 statistics.stdev(endpoints), n**0.5))
```

Steps	Sample mean	Sample standard deviation	$\sqrt{n}$
100	-0.134	9.755881	10
400	0.202	19.513892	20
1600	2.124	40.637766	40

The sampled standard deviations are close to $10, 20, 40$. Doubling $\sqrt{n}$ roughly doubles the observed spread. If $S_n = X_1 + \cdots + X_n$ with independent fair ± 1 steps, then $E(X_i) = 0$ and $\mathrm{Var}(X_i) = 1$, so $\mathrm{Var}(S_n) = n$ exactly. That calculation proves the model's standard deviation $\sqrt{n}$. The seeded runs

check the implementation on particular samples and illustrate the law. The observed discrepancies are sampling fluctuations.

Exercise 8.3. Fixed points and local stability

Hint. Solve $x = rx(1-x)$, then evaluate $f_r'(x) = r(1-2x)$.

Solution. For $r \neq 0$, the fixed points are 0 and $1 - 1/r$. For $r = 0$, only 0 is fixed. The derivatives at the two points are r and $2 - r$, respectively. The derivative criterion gives local asymptotic attraction of 0 when $|r| < 1$, and of $1 - 1/r$ when $|2 - r| < 1$, that is, $1 < r < 3$.

In the book's usual range $0 \le r \le 4$ on $[0, 1]$, the open attraction ranges are $0 \le r < 1$ and $1 < r < 3$. For $r > 1$, zero is repelling, and for $r > 3$ the nonzero fixed point is repelling. At $r = 1$ the two fixed points coincide, and at $r = 3$ the multiplier is -1; the strict derivative test is inconclusive at these boundaries.

The boundary behavior can also be determined. At $r = 1$, every sufficiently small positive x decreases under $x \mapsto x - x^2$ toward zero, giving attraction from within $[0, 1]$. At $r = 3$, put $y = x - 2/3$. One step gives $y \mapsto -y - 3y^2$ and two steps give

$$y \mapsto y - 18y^3 - 27y^4 = y\big(1 - 18y^2 - 27y^3\big).$$

For sufficiently small nonzero y, the factor lies between zero and one. The even iterates therefore shrink toward zero; the odd iterates do too. Thus this boundary fixed point is still locally attracting, but the linear contraction criterion does not prove it and convergence is slower.

Exercise 8.4. The attracting two-cycle at $r = 3.2$

Hint. Solve $f_r(f_r(x)) = x$ and remove the fixed-point factors.

Solution. The two distinct cycle values, for $r > 3$, are

$$x_{\pm} = \frac{r + 1 \pm \sqrt{(r-3)(r+1)}}{2r}.$$

At $r = 3.2$ these are approximately 0.513044509533 and 0.799455490467. Substitution gives $f_r(x_-) = x_+$ and $f_r(x_+) = x_-$, to the displayed precision. Their two-step multiplier is

$$f_r'(x_-) f_r'(x_+) = -r^2 + 2r + 4 = 0.16.$$

Its absolute value is less than one, so the cycle is locally attracting. The exact expressions identify the two-cycle, and the derivative calculation explains why nearby orbits approach it after burn-in.

Exercise 8.5. Grid resolution and burn-in

Hint. Make a two-by-two comparison: change only one setting along each direction.

Solution. The four panels below use $x_0 = 0.2$, $2.8 \le r \le 4$, and 100 retained iterates. Columns compare 300 and 1200 evenly spaced parameter values; rows compare burn-in 100 and 1000. The

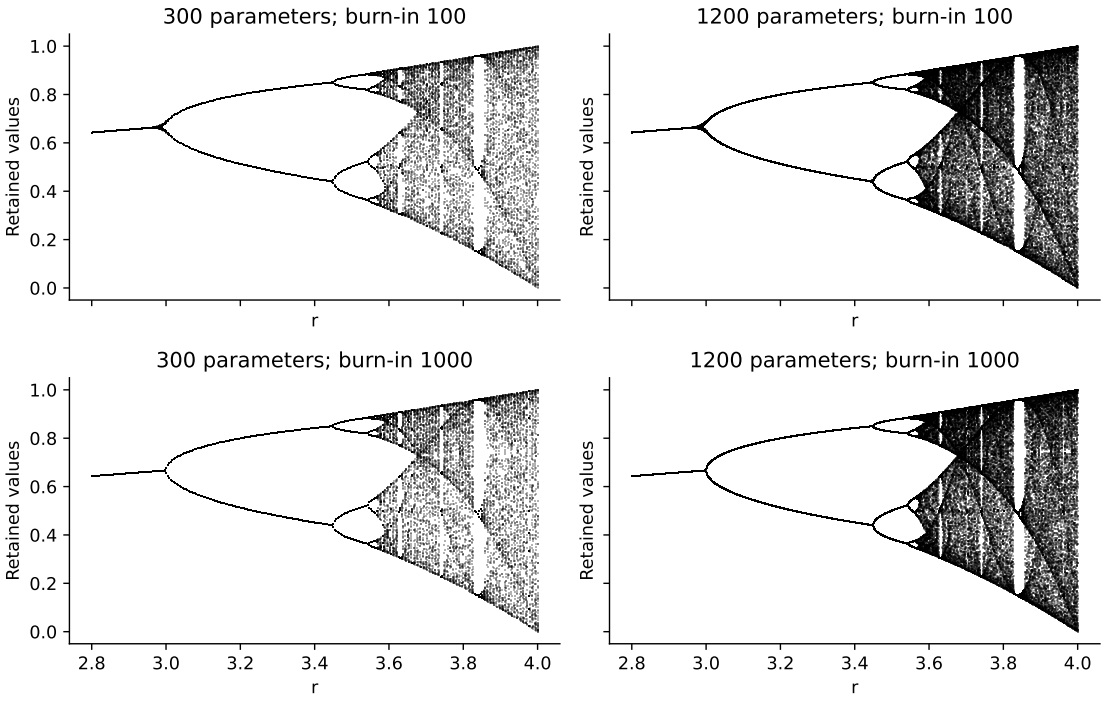


Fig. 8.1 Independent changes in parameter-grid density and discarded transient.

split near $r = 3$, subsequent branch splitting, broad irregular bands, and a prominent periodic window near $r = 3.83$ persist. A coarse grid leaves visible gaps and can miss narrow windows. Short burn-in retains slowly converging transient points near transitions; longer burn-in removes many of these extra points. A denser grid cannot itself remove transients, and longer burn-in cannot recover parameter values never sampled. Exact pixel agreement in irregular regions is not expected, and these plots do not prove a complete classification of parameter values.

Exercise 8.6. Separation of nearby initial values

Hint. Plot absolute separation and record the recurrence's evaluation order.

Solution. Use $r = 3.9$, $x_0 = 0.2$, $y_0 = 0.2 + 10^{-10}$ and 100 updates, always evaluating `r*x*(1-x)`.

```
x, y = 0.2, 0.2 + 1e-10
separation = [abs(x-y)]
for _ in range(100):
    x = 3.9*x*(1-x)
    y = 3.9*y*(1-y)
    separation.append(abs(x-y))
assert all(d >= 0 for d in separation)
```

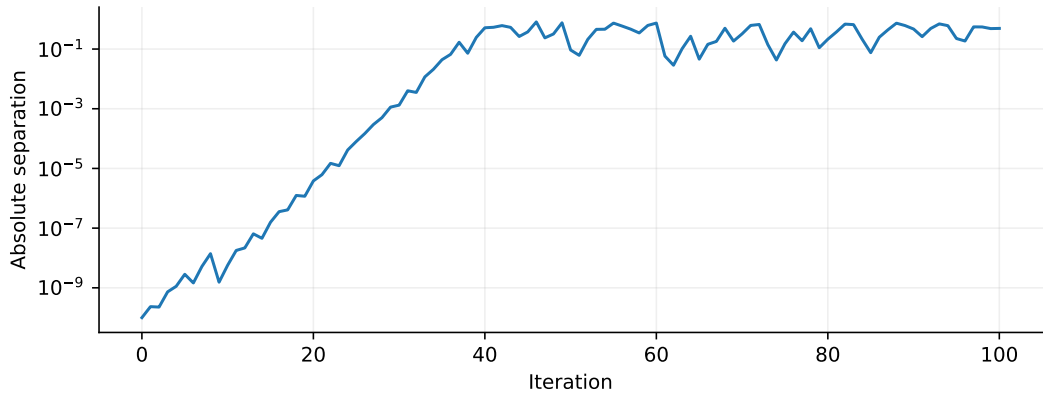


Fig. 8.2 Absolute separation on a logarithmic vertical axis for the declared 100-step binary64 calculation.

The initial discrepancy becomes much larger during this finite run, with contractions as well as growth. Later separation is limited by the bounded state interval, so growth cannot continue exponentially without bound. This is evidence of finite-time sensitivity for these inputs. It does not prove a positive limiting Lyapunov exponent or sensitivity for all initial conditions. A zero separation, if produced by rounding, must be marked explicitly or omitted with explanation on a logarithmic axis.

Further Investigation: A Worked Model

Hint. Choose a parameter where a local theorem is short.

One model response. Fix $r = 2.5$ and $x_0 = 0.2$. Discard 20, then 100 iterates in separate runs and retain the next 20. The fixed point is 0.6 and its multiplier is $2 - 2.5 = -0.5$. The negative sign predicts alternating errors; absolute value below one predicts local decay. Keep x_0, r, the formula, and both burn-in values with the data. The derivative calculation proves local attraction, while the computed trajectory shows this initial value approaching the attracting neighborhood.

Chapter 9
Graphs and Networks

Mathematical Checkpoint

Hint. Check vertex membership as well as adjacency. Decide whether repeated vertices are permitted.

Solution. Here “path” means a simple path, so no vertex repeats. The fixed input graph is assumed to be a valid undirected simple adjacency map.

```
def valid_path(graph, path, start, end):
    if not path or start not in graph or end not in graph:
        return False
    if path[0] != start or path[-1] != end:
        return False
    if any(v not in graph for v in path):
        return False
    if len(set(path)) != len(path):
        return False
    return all(v in graph[u] for u, v in zip(path, path[1:]))

graph = {0:{1}, 1:{0,2}, 2:{1}, 3:set()}
assert valid_path(graph, [0,1,2], 0, 2)
assert valid_path(graph, [3], 3, 3)
assert not valid_path(graph, [], 0, 2)
assert not valid_path(graph, [0,1], 0, 2)
assert not valid_path(graph, [0,2], 0, 2)
```

A successful check verifies this particular path. It does not compare its length with other paths. If the intended certificate permits repeated vertices, call it a walk and remove the distinctness condition explicitly.

In the displayed graph, $[0, 1, 2]$ is accepted from 0 to 2; $[0, 1]$ has the wrong endpoint and $[0, 2]$ uses a nonedge. The one-vertex path [3] is accepted from 3 to itself and has length zero. To show that the path is shortest, we also need a lower bound. Here the distinct endpoints need at least one edge, and the absent edge 02 rules out length one, so the accepted two-edge path is shortest. For a larger graph, completed breadth-first distance layers provide a corresponding lower bound.

Exercise 9.1. The handshaking identity on ten graphs

Hint. Insert isolated vertices before adding random edges.

Solution. Use vertices $0, \dots, 7$, seed 314159, and include each possible edge when one successive random value is less than 0.35.

```
import random
from itertools import combinations
import networkx as nx
rng = random.Random(314159)
rows = []
for trial in range(10):
    G = nx.Graph()
    G.add_nodes_from(range(8))
    G.add_edges_from(e for e in combinations(range(8), 2)
                     if rng.random() < 0.35)
    degree_sum = sum(d for _, d in G.degree())
    assert degree_sum == 2*G.number_of_edges()
    rows.append((trial, G.number_of_edges(), degree_sum))
```

Trial	$\lvert E\rvert$	$\sum_v \deg v$
0	15	30
1	9	18
2	10	20
3	8	16
4	8	16
5	12	24
6	10	20
7	9	18
8	13	26
9	11	22

Every row passes. The proof for all finite undirected simple graphs counts incidences (v, e): every edge has two endpoints, so $\sum_v \deg v = 2|E|$. The experiment checks the representation and calculation; it is not needed for this proof.

Exercise 9.2. Path and component certificates after an edge deletion

Hint. Delete a bridge but retain a nontrivial component.

Solution. Start with vertices $0, 1, 2, 3, 4$ and edges $01, 12, 20, 23, 34$. Delete 23. The resulting components are $S = \{0, 1, 2\}$ and $T = \{3, 4\}$. The sequence $(0, 1, 2)$ certifies a path from 0 to 2. For the component certificate, give spanning edges $01, 12$ inside S, edge 34 inside T, and check that no remaining edge joins S to T. The internal trees prove connectedness; absence of cross edges proves maximality. A bare subset with no outgoing edges could be a union of several components, so both checks matter.

Exercise 9.3. Same degrees, non-isomorphic graphs

Hint. Connectedness can distinguish graphs whose local degrees agree.

Solution. The six-cycle C_6 and the disjoint union of two triangles both have six vertices of degree two. The first is connected and the second has two components, so they are not isomorphic. An isomorphism preserves paths and hence connectedness. Thus the degree multiset is an invariant, but it is not a complete isomorphism test.

```
G = nx.cycle_graph(6)
H = nx.disjoint_union(nx.cycle_graph(3), nx.cycle_graph(3))
assert sorted(dict(G.degree()).values()) == sorted(dict(H.degree()).values())
assert not nx.is_isomorphic(G, H)
```

Exercise 9.4. Colouring C_3 through C_{10}

Hint. Alternate two colours around the cycle and inspect the closing edge.

Solution. Even cycles have chromatic number two; odd cycles have chromatic number three. On an even cycle, colour vertex i with $i \bmod 2$. On an odd cycle, alternate along vertices $0, \dots, n-2$ and give the last vertex a third colour. A two-colouring of an odd cycle would force equal colours at the two endpoints of its closing edge, a contradiction.

Cycle	χ	Two-colour assignments	Three-colour assignments
C_3	3	0	6
C_4	2	2	18
C_5	3	0	30
C_6	2	2	66
C_7	3	0	126
C_8	2	2	258
C_9	3	0	510
C_{10}	2	2	1026

The last two columns count all assignments from fixed palettes of two and three colours. For a palette of k colours the count is $(k-1)^n + (-1)^n(k-1)$, which independently checks these finite counts.

```
from itertools import product

def find_colouring(n, k):
    for colors in product(range(k), repeat=n):
        if all(colors[i] != colors[(i+1) % n] for i in range(n)):
            return colors
    return None

assert [next(k for k in range(1, 4)
             if find_colouring(n, k) is not None)
        for n in range(3, 11)] == [3,2,3,2,3,2,3,2]
```

Exercise 9.5. All labelled graphs on four vertices

Hint. There are six potential edges; use a six-bit choice.

Solution.

```
from collections import Counter
edges = list(combinations(range(4), 2))
graphs = [tuple(e for e, bit in zip(edges, bits) if bit)
          for bits in product([0, 1], repeat=6)]
assert len(graphs) == len(set(graphs)) == 64
assert Counter(map(len, graphs)) == {0:1,1:6,2:15,3:20,4:15,5:6,6:1}
```

Each subset of the six possible edges corresponds to exactly one labelled simple graph. Keep the vertex set $\{0, 1, 2, 3\}$ even for an empty edge list. The edge-count frequencies are the binomial coefficients $\binom{6}{j}$, providing an independent count check.

Exercise 9.6. Minimum spanning trees checked exhaustively

Hint. A spanning tree on n vertices uses exactly $n - 1$ edges.

Solution. Use the three weighted graphs in the following table; each has vertex set $\{0, 1, 2, 3\}$. Edge notation $01 : 2$ means weight 2 on $\{0, 1\}$.

Graph	Weighted edges	Minimum
1	$01:1, 02:4, 03:3, 12:2, 13:5, 23:6$	6
2	$01:2, 12:2, 23:2, 30:2, 02:5$	6
3	$01:-2, 02:0, 12:1, 13:3, 23:-1$	-3

For each graph enumerate all three-edge subsets, retain those that connect all four vertices, and minimize their total weight. A connected four-vertex simple graph with three edges is a tree. The valid-tree counts are $16, 8, 8$; the library minimum agrees in all three cases.

```
examples = [
 [(0,1,1),(0,2,4),(0,3,3),(1,2,2),(1,3,5),(2,3,6)],
 [(0,1,2),(1,2,2),(2,3,2),(3,0,2),(0,2,5)],
 [(0,1,-2),(0,2,0),(1,2,1),(1,3,3),(2,3,-1)]]
for weighted_edges in examples:
    G = nx.Graph()
    G.add_weighted_edges_from(weighted_edges)
    weights = []
    for selected in combinations(weighted_edges, len(G)-1):
        H = nx.Graph()
        H.add_nodes_from(G)
        H.add_weighted_edges_from(selected)
        if nx.is_connected(H):
            weights.append(sum(w for u, v, w in selected))
    tree = nx.minimum_spanning_tree(G)
    reported = sum(data['weight'] for u, v, data in tree.edges(data=True))
    assert reported == min(weights)
```

Graph 2 has several optimal trees: compare weights rather than requiring identical edge lists. A disconnected input has no spanning tree, so the specification must say how to handle it.

Further Investigation: A Worked Model

Hint. Make the domain small enough that completeness is transparent.

One model response. Test “every graph on four vertices with minimum degree at least two has a Hamilton cycle.” Enumerate the 64 labelled graphs as above. Exactly ten qualify: three four-cycles, six copies of K_4 with one edge deleted, and K_4 itself. Each has a Hamilton cycle. Retain a cyclic ordering and verify every consecutive edge, including the closing edge. The ten-case classification is an independent completeness argument. This proves the four-vertex statement. It does not extend to all sizes: $K_{2,3}$ is a connected five-vertex counterexample because a cycle in a bipartite graph alternates parts and therefore has even length.

Chapter 10
Search and Ordered Structures

As in the main book, every lattice is nonempty. An ordered set may be empty; its ideal lattice then consists of the single ideal $\varnothing$. The two main-book lattice routines reject an empty universe. The ordered-set analyzer in this companion may accept the empty ordered set, but reports that it is not a lattice.

Mathematical Checkpoint

Hint. Check the existing adjacent pair, then try each unused entry.

Solution. The prefix $(0, 2)$ passes the current adjacency test because $|0 - 2| = 2$. Its unused entries are 1 and 3, but both differ from its last entry 2 by 1. Neither can be appended, so this branch has no completion. Passing the current test is necessary, not sufficient, for extendibility. In contrast, $(0, 1)$ already contains a forbidden adjacent pair and must be rejected immediately. The two complete solutions are $(1, 3, 0, 2)$ and $(2, 0, 3, 1)$.

For the chain $0 < 1 < 2$, examine all eight subsets. Exactly $\varnothing, \{0\}, \{0, 1\}, \{0, 1, 2\}$ are downward closed. For example, $\{0, 2\}$ fails because $1 \leq 2$ but 1 is missing.

```
from itertools import product, permutations, combinations

def separated(values):
    return all(abs(a-b) != 1 for a, b in zip(values, values[1:]))

prefix = (0, 2)
assert separated(prefix)
assert not separated((0, 1))
assert all(not separated(prefix + (v,)) for v in (1, 3))
assert not any(separated(prefix + tail)
               for tail in permutations((1, 3)))
assert {p for p in permutations(range(4)) if separated(p)} == {
    (1, 3, 0, 2), (2, 0, 3, 1)
}

def ideals(elements, leq):
    answers = []
    for bits in product([0, 1], repeat=len(elements)):
        I = frozenset(x for x, bit in zip(elements, bits) if bit)
        if all(x in I for y in I for x in elements if leq(x, y)):
            answers.append(I)
```

```
    return answers

chain_ideals = ideals([0,1,2], lambda x,y: x <= y)
assert set(chain_ideals) == {frozenset(), frozenset({0}),
                             frozenset({0,1}), frozenset({0,1,2})}
```

Appending a new entry leaves every old adjacent pair in place. A prefix with adjacent consecutive values can therefore never become a separated permutation, so rejecting it removes no solution. At each accepted prefix, every unused value is tried: induction on its length shows that every permitted full permutation is reached. For the ideals, each subset has exactly one three-bit membership vector, so all $2^3 = 8$ candidates are checked once. Downward closure in a chain forces an initial segment; the four displayed initial segments prove the predicted count.

Exercise 10.1. Permutations, subsets, and binary words

Hint. Include $n = 0$ and distinguish a subset from one particular encoding of it.

Solution.

n	Permutations	Subsets	Binary words
0	1	1	1
1	1	2	2
2	2	4	4
3	6	8	8
4	24	16	16
5	120	32	32
6	720	64	64

```
from math import factorial
for n in range(7):
    ps = list(permutations(range(n)))
    words = list(product([0, 1], repeat=n))
    subsets = [frozenset(c) for k in range(n+1)
               for c in combinations(range(n), k)]
    assert len(ps) == len(set(ps)) == factorial(n)
    assert len(subsets) == len(set(subsets)) == 2**n
    assert len(words) == len(set(words)) == 2**n
```

Successive positions of a permutation have $n, n-1, \ldots, 1$ choices. A subset or binary word makes n independent two-way choices. At $n = 0$ there is one empty object of each kind.

Exercise 10.2. Safety of separated-permutation pruning

Hint. Once two adjacent entries are fixed, later appends cannot separate them.

Solution. A prefix is rejected precisely when its new last pair has absolute difference one. Any extension keeps that pair adjacent, so every completion of the rejected prefix violates the con-

dition. Hence pruning removes no valid solution. Conversely every prefix of a valid permutation has no forbidden adjacent pair; following its entries through the sorted choices reaches its leaf without pruning. Choosing each unused entry once also prevents duplicates. This proves validity, completeness, and uniqueness of the output.

Exercise 10.3. Binary words avoiding 111

Hint. Classify valid words by their final run of ones.

Solution. Let b_n count length-n words with no three consecutive ones. The initial values are $b_0 = 1, b_1 = 2, b_2 = 4$. For $n \geq 3$, a valid word ends uniquely in 0, 01, or 011. Removing that suffix gives a valid word of length $n-1, n-2$, or $n-3$, respectively; appending it reverses the operation without creating 111. The classes are disjoint and exhaustive. Thus

$$b_n = b_{n-1} + b_{n-2} + b_{n-3} \qquad (n \geq 3).$$

The first eleven values are $1, 2, 4, 7, 13, 24, 44, 81, 149, 274, 504$.

```
def no_three_ones(n):
    if type(n) is not int or n < 0:
        raise ValueError("expected integer >= 0")
    return [''.join(map(str, word))
            for word in product([0,1], repeat=n)
            if all(word[i:i+3] != (1,1,1) for i in range(n-2))]

counts = [len(no_three_ones(n)) for n in range(11)]
assert counts == [1,2,4,7,13,24,44,81,149,274,504]
assert all(counts[n] == sum(counts[n-3:n]) for n in range(3,11))
```

Exercise 10.4. The divisibility order on the divisors of 30

Hint. A cover adds exactly one missing prime factor.

Solution. The elements are $D = \{1, 2, 3, 5, 6, 10, 15, 30\}$, ordered by divisibility. Reflexivity is $a \mid a$. If $a \mid b$ and $b \mid a$ for positive integers, then $a = b$, proving antisymmetry. If $a \mid b$ and $b \mid c$, multiplying the quotient identities gives $a \mid c$, proving transitivity. The twelve cover pairs are

$$(1,2), (1,3), (1,5),$$
$$(2,6), (2,10), (3,6), (3,15), (5,10), (5,15),$$
$$(6,30), (10,30), (15,30).$$

A cover test checks $a \neq b$, $a \mid b$, and absence of a distinct $c \in D$ with $a \mid c \mid b$. Since $30 = 2 \cdot 3 \cdot 5$ is square-free, identifying a divisor with its set of prime factors identifies this order with the subset order on a three-element set.

Exercise 10.5. Extremal elements

Hint. “Minimal” excludes a smaller element; “least” requires being below all.

Solution. The minimal elements form $\{1\}$, and the maximal elements form $\{30\}$. The least element is 1, because it divides every member of D. The greatest element is 30, because every member divides it. In a general ordered set there can be several minimal elements and no least element. Here the divisibility checks establish the stronger claims directly.

Exercise 10.6. Ideals of a three-element chain and antichain

Hint. In a chain an ideal is an initial segment; in an antichain no distinct element forces another one.

Solution. For $a < b < c$, the ideals are $\varnothing, \{a\}, \{a, b\}, \{a, b, c\}$, four in all. For the antichain $\{a, b, c\}$, every subset is an ideal:

$$\varnothing, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}.$$

There are eight. Both answers include the empty ideal and the full set.

Exercise 10.7. An ordered set that is not a lattice

Hint. A missing common upper bound is already a certificate of a missing join.

Solution. Take $\{0, a, b\}$ with $0 < a$, $0 < b$, and a, b incomparable. For the pair (a, b), all common lower bounds form $\{0\}$ and all common upper bounds form $\varnothing$. Thus the meet exists and equals 0, but there is no join, so this ordered set is not a lattice. The certificate contains the pair and both complete sets of bounds. Simply reporting that no join was found would leave the reason unclear.

Exercise 10.8. Distributivity of an ideal lattice

Hint. Meet and join of ideals are intersection and union.

Solution. Use the same three-element order $0 < a, 0 < b$. Its ideals are

$$\varnothing, \{0\}, \{0, a\}, \{0, b\}, \{0, a, b\}.$$

An intersection or union of downward-closed sets is downward closed. Under inclusion, intersection is their greatest lower bound and union their least upper bound. The ordinary set identities

$$I \cap (J \cup K) = (I \cap J) \cup (I \cap K), \quad I \cup (J \cap K) = (I \cup J) \cap (I \cup K)$$

therefore prove distributivity. Exhaustively checking both equalities on all $5^3 = 125$ triples confirms the implementation.

```
I = ideals([0,1,2], lambda x,y: x == y or (x == 0 and y in [1,2]))
assert len(I) == 5
for A, B, C in product(I, repeat=3):
    assert A & (B | C) == (A & B) | (A & C)
    assert A | (B & C) == (A | B) & (A | C)
```

The set-identity proof applies to the ideal lattice of every ordered set, not only this finite example.

Part III Capstone: A Finite Ordered Set Analyzer

Hint. Validate the relation before interpreting covers or bounds.

One model response. The supplied `programs/ordered_set_analyzer.py` checks reflexivity, antisymmetry, and transitivity and retains a failing element, pair, or triple when a check fails. For a valid order it computes covers, minimal and maximal elements, least and greatest elements, and all pairwise bound sets. It returns meet and join tables when every pair has the required greatest lower and least upper bound. A missing operation is accompanied by its pair and complete set of common bounds.

For divisibility on the ordered element list $(1, 2, 3, 6)$, the least and greatest elements are $1, 6$; covers are $(1, 2), (1, 3), (2, 6), (3, 6)$. The six ideals are

$$\varnothing, \{1\}, \{1, 2\}, \{1, 3\}, \{1, 2, 3\}, \{1, 2, 3, 6\}.$$

Rows and columns in both tables follow $(1, 2, 3, 6)$:

$\wedge$	1	2	3	6
1	1	1	1	1
2	1	2	1	2
3	1	1	3	3
6	1	2	3	6

$\vee$	1	2	3	6
1	1	2	3	6
2	2	2	6	6
3	3	6	3	6
6	6	6	6	6

These are the gcd and lcm tables, an independent arithmetic check. For an invalid relation use the diagonal on $\{0, 1, 2\}$ together with $(0, 1), (1, 2)$ but omit $(0, 2)$. The transitivity witness is $(0, 1, 2)$.

For a complete small search, include every diagonal pair and choose each off-diagonal pair independently. On three labelled elements there are $2^6 = 64$ such candidate relations; exactly 19 are orders, of which six are lattices (the six labelled chains). The program also checks all candidates on one, two, and four elements, yielding respectively $1, 3, 219$ orders and $1, 2, 36$ lattices. No isomorphism reduction is used. Chains, antichains, divisibility orders, and the five-element ideal lattice above provide reference cases. The program compares its lattice conclusion with an independent bounds calculation on the complete small domain.

Chapter 11
Testing and Proof

Mathematical Checkpoint

Hint. Changing a witness is not the same as invalidating it.

Solution. For the graph with edge 01 and isolated vertex 2, reject the assignment $(0, 0, 0)$ because the endpoints of edge 01 have the same colour. The assignment $(0, 1, 0)$ is valid, as is $(0, 1, 1)$: changing the isolated vertex affects no edge. Tuples here list the colours of vertices $0, 1, 2$ in that order. The additional Boolean test below checks the chosen Python representation.

```
graph = {0:{1}, 1:{0}, 2:set()}

def valid_coloring(colors, k):
    if type(k) is not int or k < 0:
        return False
    if set(colors) != set(graph):
        return False
    if any(type(c) is not int or not 0 <= c < k
           for c in colors.values()):
        return False
    return all(colors[u] != colors[v]
               for u in graph for v in graph[u])

assert not valid_coloring({0:0, 1:1, 2:True}, 2)
assert valid_coloring({0:0, 1:1, 2:0}, 2)
assert valid_coloring({0:0, 1:1, 2:1}, 2)
assert not valid_coloring({0:0, 1:0, 2:0}, 2)
print("(0,0,0): rejected; equal colours on edge 0--1")
print("(0,1,0): accepted; (0,1,1): accepted")
```

The Boolean rejection checks the exact-integer colour contract, including an isolated vertex that no edge test visits. Changing that vertex from colour 0 to colour 1 remains valid because it has no neighbours.

Exercise 11.1. Euclid's invariant as a proof

Hint. Prove preservation and termination separately.

Solution. For nonnegative integers a, b, not both zero, start with $(u, v) = (a, b)$ and repeat $(u, v) \leftarrow (v, u \bmod v)$ while $v \neq 0$. The invariant is $\gcd(u, v) = \gcd(a, b)$. Indeed, if $u = qv + r$, an integer divides both u, v exactly when it divides both v, r. Thus the sets of common divisors, and hence the gcd, are preserved. Whenever an update is made, $0 \leq r < v$; the nonnegative second entry strictly decreases until it is zero. At termination $\gcd(u, 0) = u$, so returning u gives the original gcd. If $b = 0$ initially, the same final argument applies without an update. The pair $(0, 0)$ must be handled according to a stated convention.

Exercise 11.2. Audit the no-consecutive-ones proof

Hint. This exercise concerns the book's prohibition of 11, not Exercise 10.3's prohibition of 111.

Solution. Let a_n count length-n binary words avoiding 11. For $n \geq 2$, divide them into words ending in 0 and words ending in 1. These classes are disjoint because their last digits differ, and complete because the last digit must be one of these two values. Deleting the final 0 is a bijection from the first class to valid length-$(n-1)$ words; its inverse appends 0. In the second class the final two digits must be 01. Deleting 01 is a bijection to valid length-$(n-2)$ words; its inverse appends 01, whose initial 0 prevents a new forbidden pair. Hence $a_n = a_{n-1} + a_{n-2}$ with $a_0 = 1$ for the empty word and $a_1 = 2$ for $0, 1$. The recurrence starts at $n = 2$, so the one-letter word 1 is accounted for by the initial condition.

Exercise 11.3. A bijection for binomial symmetry

Hint. Complementation reverses subset size and is its own inverse.

Solution. For an n-element set U, map each k-subset S to $U \setminus S$. The image has $n - k$ elements and complementing again recovers S. Thus the map is a bijection and $\binom{n}{k} = \binom{n}{n-k}$ for $0 \leq k \leq n$.

```
from itertools import combinations, product, permutations
for n in range(8):
    U = frozenset(range(n))
    for k in range(n+1):
        source = {frozenset(c) for c in combinations(U, k)}
        target = {frozenset(c) for c in combinations(U, n-k)}
        assert {U-S for S in source} == target
        assert all(U-(U-S) == S for S in source)
```

These checks include $k = 0, k = n$, and the empty universe. The inverse-map argument supplies the general proof.

Exercise 11.4. Repair a false universal statement

Hint. Preserve a smallest counterexample and identify why it failed.

One model response. The claim "every positive integer has an even number of divisors" fails first at 1, whose divisor list is [1]. The corrected theorem is: a positive integer has an even number of divisors if and only if it is not a square. The obstruction is the self-paired divisor $\sqrt{n}$. Pairing d

with n/d proves both directions. This correction describes all failures rather than merely excluding the first counterexample.

Exercise 11.5. Four obligations for a finite proof

Hint. A completed run is only one part of the argument.

One model response. For the associativity search on $\{0, 1\}$:

1. Complete generator: the four table entries each have two independent choices, giving every one of 16 binary operations exactly once.
2. Correct predicate: for each of the eight triples evaluate $(x * y) * z$ and $x * (y * z)$ using the declared table indexing.
3. Actual record and checks: retain the complete distribution, the tested interpreter, and the agreement with a dictionary-based implementation.
4. Inspectable verification: retain all failures for representative tables and a transparent complete calculation for the nonexistence claims.

A witness proves that a failure count occurs. It does not prove that a missing count never occurs; the latter uses complete enumeration and a correct predicate.

Exercise 11.6. Audit suggestions about a proof

Hint. Judge the mathematical content of each suggestion, not its confidence.

One model response. Use the binomial-symmetry proof from Exercise 11.3. A request for an audit can ask whether the map is defined on the stated domain, reaches the claimed target, has an inverse, and handles boundary cases. Here is a constructed set of suggestions to classify:

- "State $0 \leq k \leq n$ and identify the fixed universe U." Valid: these make the map's domain explicit.
- "Prove that taking the complement twice returns S." Valid: this supplies the inverse and hence bijectivity.
- "Add a scatterplot of the binomial coefficients." Irrelevant to the proof's logical completeness, though possibly useful for illustration.
- "The complement argument fails when $n = 2k$ because the two families have the same size." Mistaken: a bijection can map a family to itself, and the inverse calculation still applies.

The final proof is the short argument in Exercise 11.3. The comments above were constructed for this exercise; they are not a transcript of a separate conversation.

Exercise 11.7. Assertions for bisection

Hint. Compare signs without multiplying tiny function values.

Solution. Handle computed zeros first; after every nonzero update assert that the endpoints remain ordered with opposite computed signs.

```
import math

def bisect_checked(f, left, right, tolerance=1e-12):
    if not all(math.isfinite(x) for x in [left, right, tolerance]):
        raise ValueError("finite inputs required")
    if left >= right or tolerance <= 0:
        raise ValueError("ordered interval and positive tolerance required")
    def value(x):
        y = f(x)
        if not math.isfinite(y):
            raise ValueError("nonfinite function value")
        return y
    fl, fr = value(left), value(right)
    if fl == 0: return left, (left, left), "computed zero"
    if fr == 0: return right, (right, right), "computed zero"
    if (fl < 0) == (fr < 0):
        raise ValueError("opposite signs required")
    assert left < right and (fl < 0) != (fr < 0)
    while right-left > tolerance:
        mid = left/2 + right/2
        if mid == left or mid == right:
            raise ArithmeticError("requested width below resolution")
        fm = value(mid)
        if fm == 0: return mid, (mid, mid), "computed zero"
        if (fl < 0) == (fm < 0):
            left, fl = mid, fm
        else:
            right, fr = mid, fm
        assert left < right
        assert fl != 0 and fr != 0
        assert (fl < 0) != (fr < 0)
    return left/2+right/2, (left, right), "computed bracket"

q, (a,b), status = bisect_checked(lambda x:x*x-2, 1, 2)
assert b-a <= 1e-12
assert bisect_checked(lambda x:x-1, 1, 2)[2] == "computed zero"
q, (a,b), status = bisect_checked(lambda x:1e-200*(x*x-2), 1, 2)
assert a < b
```

To obtain a rigorous enclosure, assume f is continuous throughout the initial interval and certify the true signs at every retained endpoint, for example by exact arithmetic or interval evaluation excluding zero. A computed zero can result from rounding or underflow; return a point as a rigorous root only if it is independently certified to be an exact zero. Continuity plus a reliable sign change proves existence. A derivative or monotonicity condition is needed if uniqueness is also claimed.

Exercise 11.8. A round trip for permutations and cycles

Hint. Specify the ground set and whether fixed points appear as one-cycles.

Solution. The following uses one-cycles and orders cycles by their least element. A cycle (a, b, c) means $a \mapsto b \mapsto c \mapsto a$.

```
def to_cycles(p):
    n = len(p)
    if any(type(x) is not int for x in p) or sorted(p) != list(range(n)):
        raise ValueError("expected permutation")
    seen, cycles = set(), []
    for start in range(n):
        if start in seen:
            continue
        cycle, x = [], start
        while x not in seen:
            cycle.append(x)
            seen.add(x)
            x = p[x]
        cycles.append(tuple(cycle))
    return cycles

def from_cycles(cycles, n):
    if type(n) is not int or n < 0:
        raise ValueError("expected size >= 0")
    flat = [x for cycle in cycles for x in cycle]
    if (any(not cycle for cycle in cycles)
        or any(type(x) is not int for x in flat)
        or sorted(flat) != list(range(n))):
        raise ValueError("cycles must partition the ground set")
    p = list(range(n))
    for cycle in cycles:
        for i, x in enumerate(cycle):
            p[x] = cycle[(i+1) % len(cycle)]
    return p

for n in range(8):
    for p in permutations(range(n)):
        cycles = to_cycles(p)
        assert from_cycles(cycles, n) == list(p)
        assert to_cycles(from_cycles(cycles, n)) == cycles
```

Arbitrary input cycle lists may differ by rotations or by ordering their cycles. The reverse round trip then returns a canonical equivalent form, not necessarily the identical text. The exhaustive tests above start with that canonical form. Invalid repeated or missing entries must be rejected.

Exercise 11.9. Three tests from order duality

Hint. An ideal in the dual order is an upper set in the original order.

Solution. Write P^{op} for the same elements with the order reversed. Three useful exact tests are:

1. The ideals computed for P^{op} equal the upper sets computed directly for P.
2. If I is an ideal of P, then $P \setminus I$ is an ideal of P^{op}; complementing twice recovers I. Compare the complete families under this bijection.
3. For ideals I, J of P, their complements satisfy $P \setminus (I \cup J) = (P \setminus I) \cap (P \setminus J)$ and $P \setminus (I \cap J) = (P \setminus I) \cup (P \setminus J)$ in the ideal lattice of the dual. Test the reversal of joins and meets on every pair.

Use a three-element chain, an antichain, and $0 < a, 0 < b$ as reference orders. The distinction between an upper set of P and an ideal of the dual prevents an accidental reversal of the wrong relation.

Exercise 11.10. Determinant under a row interchange

Hint. Swap two distinct rows and compare exact determinants.

Solution. For any square matrix, interchanging two rows negates its determinant. Use both a nonsingular case and a singular case.

```
import sympy as sp
for A in [sp.Matrix([[1,2],[3,4]]), sp.Matrix([[1,2],[2,4]])]:
    B = A.copy()
    B.row_swap(0, 1)
    assert B.det() == -A.det()
    B.row_swap(0, 1)
    assert B == A
```

The first determinants are -2 and 2; the second are both zero. This is a metamorphic test: we change the input in a specified way and predict how the answer will change. A zero determinant cannot reveal a sign error, so we also need the nonsingular case. The alternating multilinear definition of determinant proves the transformation rule.

Exercise 11.11. Associativity of addition modulo n

Hint. Check all ordered triples, including zeros and repeated entries.

Solution.

```
checked = 0
for n in range(2, 21):
    for x, y, z in product(range(n), repeat=3):
        assert ((x+y) % n + z) % n == (x + (y+z) % n) % n
        checked += 1
assert checked == 44099
```

All $\sum_{n=2}^{20} n^3 = 44,099$ assignments pass. The general proof is that reducing either partial sum modulo n changes it by a multiple of n, so both final values are the unique representative of $x+y+z$ modulo n in $\{0, \dots, n-1\}$. Thus associativity holds for every positive integer modulus, beyond the tested range.

Exercise 11.12. Mutate colouring certificates

Hint. Break one specified condition at a time, and also keep a changed certificate that remains valid.

Solution. Use the checkpoint's graph and checker.

```
valid = {0:0, 1:1, 2:0}
bad = [
    {0:0, 1:0, 2:0},       # equal colors on edge 0--1
    {0:0, 1:1},            # missing isolated vertex
    {0:0, 1:1, 2:2},       # outside palette
    {0:0, 1:1, 2:False},   # Boolean instead of integer
    {0:0, 1:1, 2:0, 3:0}   # extra vertex
]
assert valid_coloring(valid, 2)
assert all(not valid_coloring(c, 2) for c in bad)
assert valid_coloring({v:1-c for v,c in valid.items()}, 2)
```

The first rejection is required by the edge condition; the second omits vertex 2, even though it is isolated. The third uses a colour outside $\{0, 1\}$, the fourth violates the exact-integer representation, and the fifth assigns a colour to a vertex outside the graph. Each rejection has a specific reason in the definition. The last mutation permutes colour names and must remain valid. A checker that rejects every changed certificate would fail this control case.

Further Investigation: A Worked Model

Hint. Let a finite pattern suggest the right decomposition.

One model response. Enumerating words avoiding 11 yields $1, 2, 3, 5, 8, \ldots$. The pattern suggests a two-term recurrence, but matching values is not its proof. Classify words by their endings, establish the two bijections and initial conditions as in Exercise 11.2, then use induction to identify the count with F_{n+2}. The computer suggested the recurrence and checked its earliest consequences. The decomposition and induction prove it for all lengths without executing a program.

Chapter 12
Reproducible Investigations

Mathematical Checkpoint

Hint. A table on two elements has four cells; an associativity test has three inputs.

Solution. A tuple $t = (t_0, t_1, t_2, t_3)$ encodes $x * y = t_{2x+y}$. The Cartesian product of four copies of $\{0, 1\}$ visits each of the $2^4 = 16$ tables once. For each table, test all $2^3 = 8$ triples. The frequency dictionary increments the entry indexed by the number of failures, and a witness dictionary keeps the first table at each count.

```
from itertools import product, permutations
from collections import Counter

def failures(table, n):
    return [(x,y,z) for x,y,z in product(range(n), repeat=3)
            if table[n*table[n*x+y]+z] != table[n*x+table[n*y+z]]]

frequency, witnesses = Counter(), {}
for table in product(range(2), repeat=4):
    failed = failures(table, 2)
    operation = dict(zip(product(range(2), repeat=2), table))
    checked = [(x,y,z) for x,y,z in product(range(2), repeat=3)
               if operation[operation[x,y],z] != operation[x,operation[y,z]]]
    assert failed == checked
    frequency[len(failed)] += 1
    witnesses.setdefault(len(failed), (table, failed))
assert dict(frequency) == {0:8, 2:4, 4:2, 8:2}
assert sum(frequency.values()) == 16
```

For the witness $(0, 0, 1, 0)$, the failures are $(1, 0, 1)$ and $(1, 1, 1)$. In the first, $(1 * 0) * 1 = 1 * 1 = 0$, while $1 * (0 * 1) = 1 * 0 = 1$. In the second, $(1 * 1) * 1 = 0 * 1 = 0$, while $1 * (1 * 1) = 1 * 0 = 1$. For $x = 0$ both sides vanish; for $x = 1, z = 0$ both sides equal $1 - y$, covering the other six triples. The product construction proves completeness; the alternate indexing and hand checks test the implementation.

Exercise 12.1. A six-item reproduction note

Hint. Give each retained object one clear role.

One model response. Use the following small project layout.

Path	Purpose
`README.md`	Question, conventions, commands, and scope
`programs/`	Executable calculation and checkers
`results/`	Raw distributions and retained witnesses
`figures/`	Figures generated from the retained data
`notes/`	Specification, proof, and assistance record

A six-item reproduction note for the two-element search is:

1. Computation: associativity-failure counts for binary operations.
2. Domain: all 16 labelled tables on $\{0, 1\}$ and all eight triples for each table; no sampling or isomorphism reduction.
3. Code and environment: the supplied chapter script, Python version recorded with results, standard library only; run without `-O`.
4. Checks: complete totals, a dictionary-based second implementation, and manual evaluation of retained witnesses.
5. Preserved evidence: the frequency table and the full failing-triple list for the first table at each count.
6. Proof: the table encoding and triple substitution justify the finite reduction. The verified finite conclusion concerns two elements only.

Exercise 12.2. A reproducible seed

Hint. Create a local generator inside the function so outside calls cannot advance its state.

Solution. The following returns the complete experiment record.

```
import random

def recorded_walk(steps, seed):
    if type(steps) is not int or steps < 0:
        raise ValueError("nonnegative integer steps required")
    rng = random.Random(seed)
    path = [0]
    for _ in range(steps):
        path.append(path[-1] + rng.choice([-1,1]))
    return {"steps":steps, "seed":seed, "path":path,
            "endpoint":path[-1], "minimum":min(path),
            "maximum":max(path)}

first = recorded_walk(40, 17)
random.random()  # unrelated global activity
second = recorded_walk(40, 17)
assert first == second
assert (first["endpoint"], first["minimum"], first["maximum"]) == (4,-1,6)
```

Run the complete chapter file in a fresh process to reproduce the same record; the package check runs these scripts from separate processes. Retain the interpreter and the sampling method with the seed. Agreement establishes repeatability for the specified algorithm, parameters, and environment.

It does not establish unbiased sampling, the endpoint distribution, or the truth of a conjecture based on the sample. Using the same seed with a different sampling algorithm need not produce the same sequence.

Exercise 12.3. Disclosure of AI assistance

Hint. Say which tasks received assistance and which checks the author performed.

One model response. Substantial AI assistance was used to draft and explain portions of the investigation and to suggest tests and alternative implementations. The author reviewed the mathematical specifications, checked the proofs and retained witnesses, and reran the reported computations. The final mathematical claims and responsibility for them remain with the author.

Adapt this wording to the actual work. Include only checks the author has actually completed. For this companion, the supplied programs and their retained run records identify the checks performed during preparation.

Exercise 12.4. Material requiring authorization before sharing

Hint. Identify ownership and any existing confidentiality obligation.

One model response. Examples could include an unpublished manuscript received in confidence, a confidential referee report, another person's identifiable research data, credentials or private access tokens, and a collaborator's restricted draft. For each item, identify whose permission or applicable agreement controls disclosure. These are possible examples, not assumptions about your files. You can describe the items anonymously without reproducing their contents. Whether a particular document may be shared depends on the circumstances; this exercise makes no legal determination.

Exercise 12.5. All 27 triples for the one-failure table

Hint. Only one cell of the operation table is nonzero.

Solution. The table is

$*$	0	1	2
0	0	0	0
1	0	0	0
2	0	1	0

.

Thus $x * y = 1$ exactly when $(x, y) = (2, 1)$; otherwise it is zero. Every product $x * y$ belongs to $\{0, 1\}$, and both of those rows consist of zeros. Therefore $(x * y) * z = 0$ for every triple. The other side $x * (y * z)$ equals one exactly when $x = 2$ and $y * z = 1$, which requires $(y, z) = (2, 1)$. Hence the unique failing triple is $(2, 2, 1)$. The following table checks every triple explicitly; $L = (x * y) * z$ and $R = x * (y * z)$.

x	y	z	L	R
0	0	0	0	0
0	0	1	0	0
0	0	2	0	0
0	1	0	0	0
0	1	1	0	0
0	1	2	0	0
0	2	0	0	0
0	2	1	0	0
0	2	2	0	0
1	0	0	0	0
1	0	1	0	0
1	0	2	0	0
1	1	0	0	0
1	1	1	0	0
1	1	2	0	0
1	2	0	0	0
1	2	1	0	0
1	2	2	0	0
2	0	0	0	0
2	0	1	0	0
2	0	2	0	0
2	1	0	0	0
2	1	1	0	0
2	1	2	0	0
2	2	0	0	0
2	2	1	0	1
2	2	2	0	0

The argument explains why all 26 other triples pass. We can check that conclusion directly from the table without relying on the printed failure list.

Exercise 12.6. Relabelling the 113 associative tables

Hint. Transform both input labels and output labels by the same permutation.

Solution. If p is a permutation of $\{0, 1, 2\}$, the relabelled table t' must satisfy

$$t'_{3p(x)+p(y)} = p(t_{3x+y}).$$

Choose the lexicographically least of the six relabelled tables as a canonical representative.

```
from collections import defaultdict

def relabel(table, p):
    n = len(p)
    result = [None] * (n*n)
    for x,y in product(range(n), repeat=2):
        result[n*p[x]+p[y]] = p[table[n*x+y]]
    return tuple(result)

classes = defaultdict(list)
for table in product(range(3), repeat=9):
    if not failures(table, 3):
        canonical = min(relabel(table, p) for p in permutations(range(3)))
        classes[canonical].append(table)
assert sum(map(len, classes.values())) == 113
assert len(classes) == 24
assert Counter(map(len, classes.values())) == {1:2, 3:7, 6:15}
for representative, members in classes.items():
    orbit = {relabel(representative,p) for p in permutations(range(3))}
    assert set(members) == orbit
```

For a concrete correspondence, let $x * y = 0$ and $x \circ y = 1$ on $\{0, 1, 2\}$. The bijection $p = (1, 0, 2)$ interchanges 0 and 1 and fixes 2. Then $p(x * y) = p(0) = 1 = p(x) \circ p(y)$ for every pair. Both constant operations are associative, and this bijection places their differently labelled tables in the same class.

```
assert relabel((0,)*9, (1, 0, 2)) == (1,)*9
```

There are 24 isomorphism classes. The class-size check is $2 \cdot 1 + 7 \cdot 3 + 15 \cdot 6 = 113$. All representatives are listed below; semicolons separate table rows.

Class	Canonical table	Orbit size	Automorphisms
1	(000; 000; 000)	3	2
2	(000; 000; 001)	6	1
3	(000; 000; 002)	6	1
4	(000; 000; 012)	6	1
5	(000; 000; 222)	6	1
6	(000; 001; 002)	6	1
7	(000; 001; 012)	6	1
8	(000; 010; 002)	3	2
9	(000; 010; 222)	6	1
10	(000; 011; 011)	6	1
11	(000; 011; 012)	6	1
12	(000; 011; 022)	3	2
13	(000; 012; 012)	3	2
14	(000; 012; 021)	6	1
15	(000; 012; 222)	3	2
16	(000; 111; 222)	1	6
17	(002; 002; 002)	6	1

Class	Canonical table	Orbit size	Automorphisms
18	(002; 002; 220)	6	1
19	(002; 012; 002)	6	1
20	(002; 012; 022)	3	2
21	(002; 012; 220)	6	1
22	(011; 100; 100)	6	1
23	(012; 012; 012)	1	6
24	(012; 120; 201)	3	2

The original enumeration counts every labelled operation separately. This exercise takes equivalence classes under bijections preserving the operation. Reversing the two arguments, which forms the opposite operation, is not an additional equivalence here. Left and right projection, for instance, remain different classes. Multiplying each orbit size by its stabilizer size gives six, an additional check on the classification.

Exercise 12.7. A proposal for the next mathematical investigation

Hint. Choose a small domain you can search completely, and say which certificates will let you check the answers.

One model response. Recovering an ordered set from its ideal lattice.

The investigation will begin with a finite ordered set as a labelled universe and a reflexive relation, check that relation, and construct its lattice of order ideals. Its mathematical question is how the original order can be recovered from the nonzero join-irreducible members of that ideal lattice. The finite experiments will use at most eight elements for selected examples and will exhaust all labelled orders on at most four elements.

The program will represent downward-closed subsets by exact bit masks, list them all, ordered by inclusion, and return the union and intersection tables. A direct subset-loop implementation will serve as the reference. For an invalid relation it will return a reflexivity, antisymmetry, or transitivity witness. For a rejected ideal it will return $y \in I$ and $x \leq y$ with $x \notin I$. Every principal ideal $\downarrow x$ will be retained with the element x that generates it.

The theorem to illustrate, already part of the finite representation theory discussed in the book, is that the nonzero join-irreducible ideals are exactly the principal ideals. A finite ideal is the union of the principal ideals of its members. If it is join-irreducible, a finite-union induction forces equality with one of those principal ideals. Conversely, if $\downarrow x = J \cup K$, then $x \in J$ or $x \in K$; downward closure forces that member to be the whole principal ideal. Finally, $\downarrow x \subseteq \downarrow y$ exactly when $x \leq y$, so the recovered ordered set is isomorphic to the input.

Checks will include chains, antichains, and the divisors of 30. The complete four-element search will compare the principal-ideal map with a direct test of join-irreducibility using union. The project will retain its specification, code, environment, ordered list of candidates, certificates, full small-domain counts, and proof. One command will rerun the fast checks from a fresh copy. A short mathematical report will present the question, selected examples, complete small-domain evidence, and proof. The complete programs will be retained so that another reader can repeat the experiments. The proposed investigation explores a known theorem; it does not claim a new research result.

Part IV Capstone: A Completed Model

Hint. Keep the theorem, the executable check, and the limitations together.

One model response. The two-element associativity investigation has the following complete result.

Failures	Tables	First row-major witness
0	8	$(0,0,0,0)$
2	4	$(0,0,1,0)$
4	2	$(1,0,0,0)$
8	2	$(1,0,1,0)$

The two-failure witness was checked in the checkpoint. For $(1,0,0,0)$ the failures are $(0,0,1), (0,1,1), (1,0,0), (1,1,0)$. For $(1,0,1,0)$ the rule is $x * y = 1 - y$. Then $(x * y) * z = 1 - z$ and $x * (y * z) = z$, which differ on all eight triples. The constant-zero table is associative.

The program in the checkpoint exhausts the domain, evaluates each identity with two indexing methods, and retains the witnesses. The package's `python3 programs/run_checks.py` command reruns all chapter examples and the accompanying checks; `verify_solutions.py` also writes the complete operation distributions and witnesses. The finite theorem is that exactly $0, 2, 4, 8$ failures occur on two labelled elements, with the displayed frequencies. No one-failure table exists there. The three-element example therefore illustrates a phenomenon specific to a larger domain.

AI drafted the exposition and programs for this model. The programs were run and their checks retained; the arguments above justify the finite encodings and witnesses. This preparation record does not claim that a human author has reviewed or approved the work. For a new personal capstone, retain the same types of evidence but choose a different question, as the book requests.

Appendix A
Python: An Optional Introduction

The numbering here continues through the three exercise groups in the book: A.1–A.5, Reading and Altering; A.6–A.10, Mathematical Programs; and A.11–A.15, Testing and Generated Code.

Exercise A.1. Validate the rectangle function

Hint. Preserve both returned quantities: area and perimeter.

Solution. Use finite positive real lengths, represented here by Python integers or floats; reject Booleans and nonfinite floats.

```
import math

def rectangle_data(length, width):
    for x in (length, width):
        if type(x) not in (int, float):
            raise ValueError("numeric lengths required")
        if x <= 0 or (type(x) is float and not math.isfinite(x)):
            raise ValueError("finite positive lengths required")
    return length*width, 2*(length+width)

assert rectangle_data(4, 4) == (16, 16)
assert rectangle_data(3, 7) == (21, 20)
for bad in [0, -2, float('nan'), float('inf'), True]:
    try:
        rectangle_data(bad, 3)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid length accepted")
```

The square has area 16 and perimeter 16; the 3-by-7 rectangle has area 21 and perimeter 20. Units are square units and units, respectively. With integer lengths the calculations are exact. With floating-point lengths, the results are subject to the usual range limits and rounding.

Exercise A.2. The sign function

Hint. Test a negative number, zero, and a positive number.

Solution. For an integer input, use three mutually exclusive branches.

```
def sign(n):
    if type(n) is not int:
        raise ValueError("integer required")
    if n < 0:
        return -1
    if n == 0:
        return 0
    return 1

assert [sign(n) for n in [-7, 0, 12]] == [-1, 0, 1]
```

The order trichotomy proves that exactly one branch applies. A version for real-valued floating inputs should explicitly reject NaN, which satisfies neither the ordinary less-than nor equality comparisons in the intended way.

Exercise A.3. An accumulating sum and invariant

Hint. Before adding k, identify exactly which terms have already been added.

Solution.

```
def sum_to(n):
    if type(n) is not int or n < 0:
        raise ValueError("nonnegative integer required")
    total = 0
    for k in range(1, n+1):
        total += k
    return total

for n in range(1001):
    assert sum_to(n) == n*(n+1)//2
```

Before the iteration indexed by k, the invariant is `total` $= 1 + \cdots + (k-1) = (k-1)k/2$. It holds initially at $k = 1$. Adding k gives $k(k+1)/2$, the form required at the next iteration. After the last update the total is $n(n+1)/2$. At $n = 0$, the loop is empty and returns zero. Integer division is exact because one of $n, n+1$ is even. All 1001 declared inputs pass.

Exercise A.4. Divisors through the square root

Hint. Save paired large divisors separately and reverse that list once.

Solution.

```
from math import isqrt

def divisors(n):
    if type(n) is not int or n < 1:
        raise ValueError("positive integer required")
    small, large = [], []
    for d in range(1, isqrt(n)+1):
        if n % d == 0:
            small.append(d)
            if d*d != n:
                large.append(n//d)
    return small + large[::-1]

assert divisors(1) == [1]
assert divisors(9) == [1,3,9]
assert divisors(12) == [1,2,3,4,6,12]
assert divisors(36) == [1,2,3,4,6,9,12,18,36]
for n in range(1, 1001):
    assert divisors(n) == [d for d in range(1,n+1) if n%d == 0]
```

The small divisors arrive in increasing order, while their partners n/d arrive in decreasing order. Every divisor larger than $\sqrt{n}$ is the partner of a smaller one. A square's middle divisor is appended only once. These facts prove that the list is complete, ordered, and free of duplicates for every permitted input.

Exercise A.5. Triangular numbers in two ways

Hint. State whether the first term is T_0 or T_1.

Solution. Here the first n triangular numbers mean $T_1, \ldots, T_n$; $n = 0$ gives an empty list.

```
def triangular_formula(n):
    if type(n) is not int or n < 0:
        raise ValueError("nonnegative integer required")
    return [k*(k+1)//2 for k in range(1,n+1)]

def triangular_loop(n):
    if type(n) is not int or n < 0:
        raise ValueError("nonnegative integer required")
    result, total = [], 0
    for k in range(1,n+1):
        total += k
        result.append(total)
    return result

assert triangular_loop(5) == [1,3,6,10,15]
assert all(triangular_formula(n) == triangular_loop(n) for n in range(101))
```

Exercise A.3 proves the equality at each index. Returning all T_0 through T_n instead would give $n + 1$ terms and would answer a different question.

Exercise A.6. Relations with failure witnesses

Hint. Return the negation of the defining implication when a property fails.

Solution. The following reports `None` for a passing property and an explicit witness for a failed one. Inputs are a list of distinct hashable elements and a set of pairs on that universe.

```
from itertools import product

def relation_report(elements, relation):
    X = list(elements)
    if len(set(X)) != len(X):
        raise ValueError("distinct elements required")
    R = set(relation)
    if any(len(pair) != 2 or pair[0] not in X or pair[1] not in X
           for pair in R):
        raise ValueError("relation must consist of pairs on X")
    return {
      "reflexivity": next(((x,) for x in X if (x,x) not in R), None),
      "symmetry": next(((x,y) for x,y in product(X,repeat=2)
                        if (x,y) in R and (y,x) not in R), None),
      "antisymmetry": next(((x,y) for x,y in product(X,repeat=2)
                            if x != y and (x,y) in R and (y,x) in R), None),
      "transitivity": next(((x,y,z) for x,y,z in product(X,repeat=3)
                            if (x,y) in R and (y,z) in R and (x,z) not in R), None)
    }

X = [0,1,2]
R = {(0,0),(1,1),(2,2),(0,1),(1,2)}
r = relation_report(X, R)
assert r == {"reflexivity":None, "symmetry":(0,1),
             "antisymmetry":None, "transitivity":(0,1,2)}
assert relation_report([0,1], {(0,1),(1,0)})["antisymmetry"] == (0,1)
assert relation_report([0], set())["reflexivity"] == (0,)
```

The witness $(0, 1, 2)$ certifies transitivity failure because $(0, 1)$ and $(1, 2)$ are present but $(0, 2)$ is absent. The function checks all defining cases if it finds no failure. A relation is an order precisely when reflexivity, antisymmetry, and transitivity pass; symmetry is not required. Empty universes make all four universal conditions vacuously true.

Exercise A.7. Extend `degree_report` and test its contract

Hint. Preserve the edge input before counting it, especially when it is an iterator.

Solution. Retain the supplied function's name, validation, and return value. Convert the edge input to a list once, then add the handshaking assertion. Each accepted edge joins two distinct known vertices and contributes one at each end, so the sum of the degrees is twice the number of edges.

```
def degree_report(vertices, edges):
    vertices = list(vertices)
    edges = list(edges)  # Preserve a one-pass iterator.
    if len(set(vertices)) != len(vertices):
        raise ValueError("vertices must be distinct")
    degree = {v: 0 for v in vertices}
    seen = set()

    for u, v in edges:
        if u not in degree or v not in degree:
            raise ValueError("unknown vertex in an edge")
        if u == v:
            raise ValueError("loops are not permitted")
        edge = frozenset((u, v))
        if edge in seen:
            raise ValueError("repeated undirected edge")
        seen.add(edge)
        degree[u] = degree[u] + 1
        degree[v] = degree[v] + 1

    assert sum(degree.values()) == 2 * len(edges)
    maximum = max(degree.values()) if degree else 0
    largest = [v for v in vertices if degree[v] == maximum]
    return degree, largest

# Isolated vertices remain in the degree dictionary.
assert degree_report([0, 1, 2, 3], [(0, 1), (1, 2)]) == (
    {0: 1, 1: 2, 2: 1, 3: 0}, [1]
)
# Retain all vertices tied for maximum degree.
assert degree_report([0, 1, 2], [(0, 1), (1, 2), (2, 0)]) == (
    {0: 2, 1: 2, 2: 2}, [0, 1, 2]
)
assert degree_report([], []) == ({}, [])
assert degree_report([0, 1], []) == ({0: 0, 1: 0}, [0, 1])
# The assertion must also work with a one-pass edge iterator.
edge_iterator = iter([(0, 1), (1, 2)])
assert degree_report([0, 1, 2], edge_iterator) == (
    {0: 1, 1: 2, 2: 1}, [1]
)

def expect_degree_error(vertices, edges):
    try:
        degree_report(vertices, edges)
    except ValueError:
        return
    raise AssertionError("invalid graph was accepted")

expect_degree_error([0, 1], [(0, 1), (1, 0)])
expect_degree_error([0, 1], iter([(0, 1), (1, 0)]))
expect_degree_error([0, 1], [(0, 1), (0, 1)])
expect_degree_error([0, 1], [(0, 2)])
expect_degree_error([0], [(0, 0)])
expect_degree_error([0, 0], [])
expect_degree_error([], [(0, 1)])
```

Isolated vertices and tied maxima are permitted. The empty graph is also permitted: its degree sum and edge count are both zero, and it has no maximum-degree vertices. In a nonempty edgeless graph every vertex has maximum degree zero. A reversed duplicate is the same undirected edge,

so it is rejected, as are repeated edges, unknown endpoints, loops, and duplicate vertex labels. An edge iterator is accepted; materializing it before the loop prevents the later edge count from seeing an exhausted iterator. Each of these tests checks a requirement of the stated simple-graph specification.

Exercise A.8. Fibonacci residues and the first repeated pair

Hint. Reduce both the initial state and each updated state modulo m.

Solution.

```
def fibonacci_mod(count, m):
    if type(count) is not int or count < 0 or type(m) is not int or m <= 0:
        raise ValueError("count >= 0 and modulus > 0 required")
    a, b, values = 0, 1 % m, []
    for _ in range(count):
        values.append(a)
        a, b = b, (a+b) % m
    return values

def first_repeated_pair(m):
    if type(m) is not int or m <= 0:
        raise ValueError("positive integer modulus required")
    a, b, index, seen = 0, 1 % m, 0, {}
    while (a,b) not in seen:
        seen[a,b] = index
        a,b = b,(a+b) % m
        index += 1
    return (a,b), seen[a,b], index

assert fibonacci_mod(8, 3) == [0,1,1,2,0,2,2,1]
assert first_repeated_pair(3) == ((0,1), 0, 8)
assert first_repeated_pair(1) == ((0,0), 0, 1)
```

The return records the pair, its first index, and its repeated index. The invertibility argument in Exercise 7.2 proves that the first repeated pair is the initial pair, so there is no transient. Modulus one requires the initial pair $(0, 0)$, not $(0, 1)$.

Exercise A.9. All solutions of a cube congruence

Hint. Search a complete residue system and return every matching representative.

Solution.

```
def cube_solutions(a, m):
    if type(a) is not int or type(m) is not int or m <= 0:
        raise ValueError("integer a and positive integer m required")
    return [x for x in range(m) if pow(x, 3, m) == a % m]

assert cube_solutions(1, 7) == [1,2,4]
assert cube_solutions(6, 7) == [3,5,6]
assert cube_solutions(0, 9) == [0,3,6]
```

```
assert cube_solutions(1, 9) == [1,4,7]
assert cube_solutions(2, 9) == []
assert cube_solutions(5, 1) == [0]
```

The residue lists in Exercises 2.3 and 3.2 hand-check these examples. Each returned x represents all integers congruent to it modulo m. An empty list means the congruence has no solution, not that the search stopped early. Testing all m representatives proves completeness.

Exercise A.10. Pythagorean triples through hypotenuse 100

Hint. Keep the order $a \le b$ to avoid counting a leg swap twice.

Solution.

```
def pythagorean_triples(bound):
    if type(bound) is not int or bound < 1:
        raise ValueError("positive integer bound required")
    return [(a,b,c) for c in range(1,bound+1)
            for a in range(1,c+1) for b in range(a,c+1)
            if a*a+b*b == c*c]

triples = pythagorean_triples(100)
assert len(triples) == len(set(triples)) == 52
assert (3,4,5) in triples and (60,80,100) in triples
```

There are 52 triples, including nonprimitive multiples. Every allowed triple occurs in exactly one choice of the three loop indices, so the search is complete and duplicate free. An independent description uses the following 16 primitive triples:

Primitive triple	Multiples	Primitive triple	Multiples
$(3,4,5)$	20	$(5,12,13)$	7
$(8,15,17)$	5	$(7,24,25)$	4
$(20,21,29)$	3	$(12,35,37)$	2
$(9,40,41)$	2	$(28,45,53)$	1
$(11,60,61)$	1	$(16,63,65)$	1
$(33,56,65)$	1	$(48,55,73)$	1
$(13,84,85)$	1	$(36,77,85)$	1
$(39,80,89)$	1	$(65,72,97)$	1

For each primitive triple (a,b,c) include the multiples $k(a,b,c)$ with $1 \le k \le \lfloor 100/c \rfloor$. The sum of these multiplicities is 52 and produces the same set. The full 52-triple list is retained in `results/verified_results.json`.

Exercise A.11. An off-by-one error

Hint. Test the smallest permitted input before a typical input.

Solution. Suppose the sum function incorrectly uses `range(1,n)`. For $n = 0$ it accidentally returns the correct zero. The smallest nonnegative failure is $n = 1$: it returns zero instead of one.

```
def broken_sum(n):
    total = 0
    for k in range(1,n):
        total += k
    return total

assert broken_sum(0) == 0
assert broken_sum(1) != 1
assert sum_to(1) == 1  # repaired function from A.3
```

Changing the stop value to $n+1$ repairs the error; preserve $n = 1$ as a regression test. The invariant in Exercise A.3 proves the corrected loop.

Exercise A.12. Expand a compact loop

Hint. Translate each generator clause into one ordinary loop or conditional.

One model response. A compact proposal for positive divisors is `[d for d in range(1,n+1) if n\%d == 0]`. Written with an ordinary loop, it becomes:

```
def divisors_expanded(n):
    if type(n) is not int or n < 1:
        raise ValueError("positive integer required")
    result = []
    for d in range(1,n+1):
        if n % d == 0:
            result.append(d)
    return result

for n in range(1,1001):
    compact = [d for d in range(1,n+1) if n % d == 0]
    assert compact == divisors_expanded(n) == divisors(n)
```

The declared complete comparison domain is $1 \le n \le 1000$. It includes the smallest input, squares, primes, and composites and is small enough for the direct loop. Reading the comprehension in this order explains agreement for every positive integer. This gives a model answer to the requested code-revision exchange.

Exercise A.13. Optimize primality with the same contract

Hint. After handling two, an even divisor never needs to be tested.

Solution. The contract accepts integers, including negatives, returning false below two; nonintegers and Booleans raise `ValueError`.

```
def is_prime_reference(n):
    if type(n) is not int:
        raise ValueError("integer required")
    return n >= 2 and all(n % d != 0 for d in range(2,n))

def is_prime(n):
    if type(n) is not int:
        raise ValueError("integer required")
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    return all(n % d != 0 for d in range(3,isqrt(n)+1,2))

assert all(is_prime(n) == is_prime_reference(n) for n in range(10001))
assert [is_prime(n) for n in [-3,0,1,2,3,4,9,25,49]] == [False,False,False,True,True,False,
    False,False,False]
for bad in [True, 3.0, "3"]:
    for function in [is_prime, is_prime_reference]:
        try:
            function(bad)
        except ValueError:
            pass
        else:
            raise AssertionError("invalid input accepted")
```

Every composite has a factor no greater than its square root. An odd composite has an odd such factor. This proves that the optimized search misses none. All 10,001 integers from zero through 10,000 agree with the reference; the invalid-input checks confirm that both versions reject the same disallowed inputs.

Exercise A.14. Ten seeded 100-step walks

Hint. Return the full path and calculate the summary from that same path.

Solution.

```
import random

def walk_report(steps, seed):
    if type(steps) is not int or steps < 0:
        raise ValueError("nonnegative integer steps required")
    rng = random.Random(seed)
    path = [0]
    for _ in range(steps):
        path.append(path[-1] + rng.choice([-1,1]))
    return {"seed":seed, "path":path, "endpoint":path[-1],
            "minimum":min(path), "maximum":max(path)}

reports = [walk_report(100, seed) for seed in range(10)]
assert all(len(row['path']) == 101 for row in reports)
assert reports == [walk_report(100, seed) for seed in range(10)]
```

Seed	Endpoint	Minimum	Maximum
0	0	-1	9
1	8	-3	10
2	0	-4	18
3	10	-5	10
4	8	-4	8
5	-2	-14	2
6	12	-4	12
7	-2	-17	1
8	0	-6	3
9	-20	-23	3

The recorded seeds are exactly $0, 1, \ldots, 9$. Minimum and maximum include the starting zero. The complete paths are returned by the program; the verification package also retains the ten paths. These ten samples are not enough to establish the distribution law; the variance calculation in Exercise 8.2 supplies the theoretical result.

Exercise A.15. A one-paragraph computational report

Hint. Describe the computation, explain the checks, and state both what the evidence establishes and what remains open.

One model response. The program enumerated every integer triple satisfying $1 \leq a \leq b \leq c \leq 100$ and retained the 52 triples with $a^2 + b^2 = c^2$. The checks verified the equation and bounds for each retained triple, excluded duplicates, and compared the result with the multiples of the 16 primitive triples in the same range. The nested-loop bounds show that every permitted triple was examined, so the result is complete for this finite domain. It does not classify triples with hypotenuse greater than 100; an unrestricted classification requires the general parameterization and its proof.